INFRASTRUCTURE & DEVOPS

Linux & the Command Line

Operate confidently on any Linux box you did not build — shell, systemd, SSH, networking, and the triage runbooks an FDE lives on.

linux · bash · shell · systemd · ssh · networking · permissions · processes · awk · grep · troubleshooting · sysadmin · devops · fde

Study Pack · Comprehensive reference & interview prep · ~71 min read · 15,651 words

CONTENTS

1. Why the Command Line Is the FDE's Primary Instrument
2. The Filesystem Hierarchy and How to Navigate It
3. Files, Links, Globbing, find, and the xargs Pattern
4. Users, Groups, Permissions, sudo, and ACLs
5. Processes, Signals, Job Control, and Inspection
6. systemd: Units, systemctl, journalctl, and Timers
7. Package Management Across Distros and Air-Gapped Installs
8. SSH Deep Dive
9. Text Processing: grep, sed, awk, cut, sort, jq
10. Pipes, Redirection, File Descriptors, and Here-Docs
11. Bash Scripting That Does Not Break
12. Networking From the Shell
13. Performance and Resource Triage
14. Logs: Where They Live and How to Read Them
15. Working Remotely for Days: tmux, Dotfiles, Persistence
16. The FDE Field Kit
17. Interview Questions
18. Practice Drills and a 14-Day Fluency Plan

Why the Command Line Is the FDE's Primary Instrument

A Forward Deployed Engineer is dropped into someone else's infrastructure with a laptop, an SSH key, and a deadline. There is no IDE on the jump host, no Docker Desktop, no browser that can reach the internet. What there is — always — is a shell. Fluency here is not a nice-to-have; it is the difference between diagnosing a broken deployment in eleven minutes and escalating to a customer's platform team and losing three days.

~60	0	1973	2h
COMMANDS THAT COVER 95% OF WORK	GUIS GUARANTEED ON A CUSTOMER BOX	YEAR PIPES SHIPPED IN UNIX V3	TYPICAL TIME UNTIL THE DEMO

What actually happens on a customer engagement

You get a bastion host. It runs a distro you did not choose — RHEL 8 at a bank, Ubuntu 22.04 at a startup, SLES 15 at a manufacturer, Amazon Linux 2023 in an AWS account. Your account has no root. Outbound traffic goes through a proxy that requires a corporate CA bundle. The container registry is an internal Artifactory mirror. The Python package index is a Nexus proxy that lags upstream by six weeks. Nothing you learned from a laptop-only workflow transfers cleanly. What transfers is the shell.

COMPOSABILITY

Every tool reads text on stdin and writes text on stdout. That contract means you can assemble a diagnostic that nobody wrote in advance — `journalctl -u api --since -1h | grep 502 | awk '{print $NF}' | sort | uniq -c | sort -rn` — in twenty seconds.

UBIQUITY

Bash, coreutils, grep, sed and awk are present on essentially every Linux host built since 1995. Your fancy TUI tool is not. Learn the baseline, then layer niceties on top only where you control the image.

REPRODUCIBILITY

A command is a record. You can paste it into the customer's ticket, into a runbook, into a CI job. A sequence of mouse clicks in a web console is an anecdote.

⚠ **Assume you are a guest**

Everything you type on a customer system is potentially logged, audited, and attributed to you. Never run a destructive command you have not read twice. Never `curl | sudo bash` on someone else's box. Prefer read-only investigation until you have explicit written approval to change state.

Shell vs terminal vs console — get the vocabulary right

Term	What it actually is	Why you care
Terminal emulator	The GUI app (iTerm2, Windows Terminal, GNOME Terminal) that draws characters and handles key input	Controls colour, scrollback, and whether `Alt` sends Meta
TTY / PTY	Kernel device pair providing the character stream. `tty` prints yours; `ls /dev/pts` lists them	Scripts behave differently without a TTY — that is why Docker needs `-t`
Shell	The program interpreting your commands: bash, zsh, dash, ksh, ash (BusyBox)	`/bin/sh` on Debian is **dash**, not bash. Bash-isms silently fail
Console	The physical/virtual system console (`/dev/console` , Ctrl+Alt+F2)	Where kernel panics go when SSH is already dead

Login, non-login, interactive, non-interactive

This quadrant explains ninety percent of "it works when I SSH in but not from cron."

Invocation	Files read	Typical case
Interactive login	`/etc/profile`, then first of `~/.bash_profile` / `~/.bash_login` / `~/.profile`	`ssh host`
Interactive non-login	`/etc/bash.bashrc`, `~/.bashrc`	Opening a new tab locally, `bash` inside a session
Non-interactive	Only `$BASH_ENV` if set — **not** `.bashrc`	`ssh host 'cmd'`, cron, systemd, CI

✦ **The canonical fix**
Put environment (PATH, exports) in `~/.profile`; put interactive niceties (aliases, prompt) in `~/.bashrc`; have `~/.bash_profile` contain `[ -f ~/.bashrc ] && . ~/.bashrc`. When cron fails with "command not found", it is almost always because cron's PATH is `/usr/bin:/bin` and nothing else.

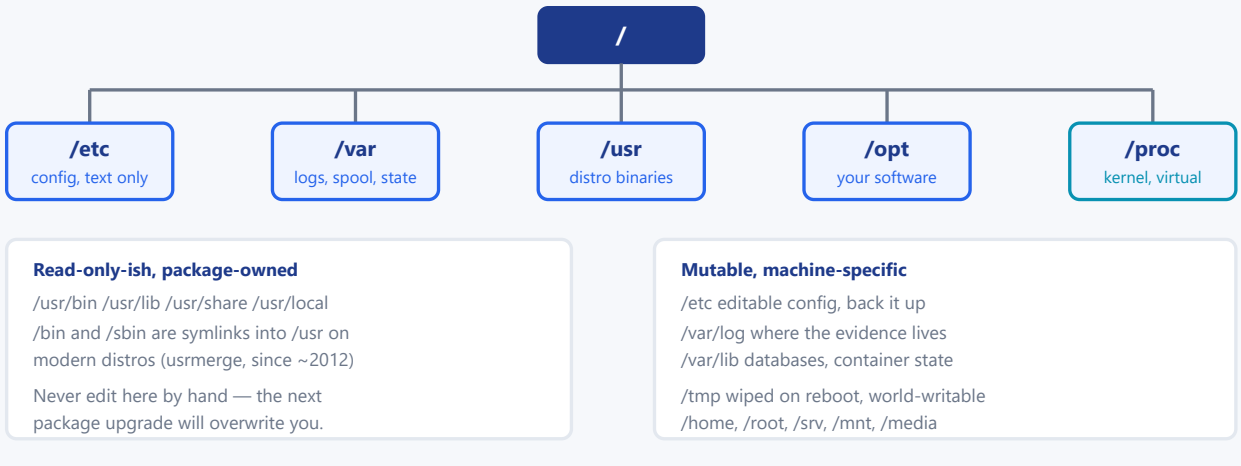
Readline: the keystrokes that compound

Keys	Action	Keys	Action
`Ctrl`+`R`	Reverse-search history (press again to cycle)	`Ctrl`+`A` / `E`	Start / end of line
`Ctrl`+`W`	Delete previous word	`Ctrl`+`U` / `K`	Cut to start / end
`Ctrl`+`Y`	Paste what you cut	`Alt`+`.`	Insert last argument of previous command
`Ctrl`+`L`	Clear screen (keeps the line)	`Ctrl`+`X` `Ctrl`+`E`	Open current line in `$EDITOR`

History expansion earns its keep too: `!!` repeats the last command, `sudo !!` repeats it with privilege, `!$` is the last argument, `^old^new` re-runs with a substitution. Set `HISTCONTROL=ignoreboth` and `HISTSIZE=50000` so your history is actually useful.

The Filesystem Hierarchy and How to Navigate It

Linux has one tree. Everything — disks, processes, kernel knobs, USB devices — is a file somewhere under `/`. The Filesystem Hierarchy Standard (FHS) tells you where to look before you have to search, and on a customer's unfamiliar host that knowledge is worth minutes you do not have.



The FHS top level: immutable distro content on the left, machine state on the right.

Directory-by-directory, with the FDE angle

Path	Contents	What you do there on day one
`/etc`	System-wide configuration, all plain text	`/etc/os-release`, `/etc/hosts`, `/etc/resolv.conf`, `/etc/ssl/certs`, `/etc/systemd/system`
`/var/log`	Logs: `syslog` / `messages`, `auth.log` / `secure`, app dirs, `journal/`	First place you look after a failed deploy
`/var/lib`	Persistent service state: `docker/`, `containerd/`, `postgresql/`, `kubelet/`	Where the disk actually went when `/` fills up
`/usr/local`	Software compiled or installed outside the package manager	Safe place to drop a static binary you brought with you
`/opt`	Self-contained third-party packages, one directory each	Corporate agents live here — CrowdStrike, Splunk, Dynatrace
`/proc`	Virtual filesystem exposing kernel & process state	`/proc/cpuinfo`, `/proc/meminfo`, `/proc/<pid>/environ`, `/proc/<pid>/limits`
`/sys`	Device/driver model, cgroups v2 knobs	`/sys/fs/cgroup/memory.max` inside containers
`/dev`	Device nodes plus `null`, `zero`, `urandom`, `/dev/tcp` (bash built-in)	`echo > /dev/tcp/host/443` tests reachability with no tools installed
`/run`	tmpfs for runtime data: PID files, sockets	`/run/docker.sock`, `/run/systemd/`

✦ **Identify the host in three commands**

`cat /etc/os-release` for distro and version, `uname -r` for kernel, `hostnamectl` for a combined view including virtualization type. If `hostnamectl` says `Virtualization: kvm` you are on a VM; `docker` or `wsl` tells you something entirely different about what you can expect to work.

Navigating without wasting keystrokes

BASH

```bash
# Jump back to where you were
cd -                        # toggle between last two directories
pushd /var/log; popd        # directory stack when you need three or four

# Where am I really? (resolves symlinks)
pwd -P

# Tree view without installing tree
find . -maxdepth 2 -type d | sort

# What is this file?
file /usr/bin/python3
stat -c '%A %U:%G %s bytes  mtime=%y  %n' /etc/hosts

# Biggest things under here, fast
du -xh --max-depth=1 . 2>/dev/null | sort -h | tail -15
```

The `-x` on `du` keeps it on one filesystem, which stops you from walking an NFS mount for four minutes. `2>/dev/null` suppresses the permission-denied noise you will get as a non-root user.

Files, Links, Globbing, find, and the xargs Pattern

Finding things is the core FDE skill. The customer says "the config is somewhere under `/opt`"; you have five minutes. `find` plus `xargs` is the instrument that answers almost any "where is it / which ones match / do this to all of them" question.

Hard links vs symbolic links

A hard link is another directory entry pointing at the same inode — same file, two names, no "original". A symlink is a tiny file containing a path. Delete the target and the symlink dangles; delete one hard link and the data survives until the last link goes.

BASH

```
ln  /data/model.bin  /opt/app/model.bin      # hard link: same inode, same filesystem
ln -s /data/model.bin /opt/app/model.bin     # symlink: a pointer, can cross filesyste
ls -li                                        # first column is the inode number
readlink -f /opt/app/current                  # resolve a chain of symlinks fully
```

> **The blue/green symlink trick**
> Deploy releases to `/opt/app/releases/2026-03-11T0930` and point `/opt/app/current` at it with `ln -sfn`. Rollback becomes one atomic symlink swap plus a service restart — no copy, no downtime window, no "which version is live?" ambiguity. The `-n` matters: without it, `ln -sf` creates a link *inside* the existing directory symlink.

Globbing is the shell, not the command

BASH

```
*.log            # any chars
report-?.csv     # exactly one char
log-[0-9][0-9]   # character class
{api,worker}.yaml   # brace expansion (not a glob - expands even if no file exists)

shopt -s globstar
ls **/*.py       # recursive, bash 4+

shopt -s nullglob  # unmatched glob expands to nothing instead of the literal string
shopt -s dotglob   # include hidden files
```

If a glob matches nothing, bash by default passes the literal pattern to the command — which is why `rm *.bak` in an empty directory complains about a file named `*.bak`. In scripts, `nullglob` plus an explicit emptiness check avoids a whole class of bugs.

find: the twelve expressions worth memorising

BASH

```
find /opt -name '*.yaml'                      # by name (quote it! or the shell expands
find /opt -iname 'Config*'                    # case-insensitive
find /var/log -type f -size +100M             # files over 100 MB
find /var/log -type f -mtime -1               # modified in the last 24 h
find /var/log -type f -mmin -15               # modified in the last 15 minutes
find / -xdev -type f -perm -4000 2>/dev/null # setuid binaries, one filesystem
find . -type d -name node_modules -prune -o -type f -print   # skip a subtree
find . -newer /tmp/marker                     # changed since a reference file
find /data -type f -user svc_app -group app  # ownership
find . -empty -type f                         # zero-byte files
find . -name '*.tmp' -delete                  # built-in delete, no xargs needed
find . -type f -printf '%s\t%p\n' | sort -rn | head   # GNU-only: size then path
```

> ⚠ **BSD vs GNU find**
>
> `-printf`, `-mmin` and `-delete` are GNU extensions. macOS and some minimal images ship BSD or BusyBox find where they do not exist. On a customer box, test with a harmless case before relying on a flag in a script.

The xargs pattern

`find ... -print0 | xargs -0` is the safe idiom: NUL-separated so filenames with spaces, quotes or newlines cannot break the pipeline.

BASH

```
# Grep only inside YAML files, safely, in parallel
find /opt/app -name '*.yaml' -print0 | xargs -0 grep -l 'imagePullSecrets'

# -n1 one argument per invocation; -P4 four processes in parallel
find . -name '*.json' -print0 | xargs -0 -n1 -P4 jq -e . >/dev/null

# -I{} when the argument is not last
find /backup -name '*.sql.gz' -print0 | xargs -0 -I{} sh -c 'echo "== {}"; zcat {} | h

# Preview before you destroy: -t prints each command, -p prompts
find /tmp -name 'core.*' -print0 | xargs -0 -t rm -f
```

`find -exec cmd {} +` does the same batching without a pipe and is marginally safer because there is no quoting hand-off: `find . -name '*.log' -exec gzip {} +`. Use `\;` instead of `+` when the command can only take one file at a time.

locate, which, type, and friends

BASH

```bash
sudo updatedb && locate nginx.conf   # instant, but only as fresh as the database
type -a python3                       # shows aliases, functions, AND all PATH hits
command -v kubectl                    # POSIX, script-safe "is it installed?"
which -a java                         # all matches on PATH
readlink -f "$(command -v java)"      # the real binary behind the alternatives symlin
ldd /usr/bin/curl                     # shared library dependencies
strings /opt/vendor/agent | grep -i version   # when there is no --version flag
```

✦ **Use `command -v` in scripts**

`which` is an external binary with inconsistent exit codes across distros, and is absent from some minimal images. `command -v tool >/dev/null || { echo "need tool"; exit 1; }` is portable and correct.

Users, Groups, Permissions, sudo, and ACLs

On a customer system you will almost never be root, and that constraint shapes everything. Knowing precisely what an octal mode means, what `sudo -l` tells you about your own blast radius, and why your service can read a file but not the directory containing it, is non-negotiable.

Reading `ls -l` correctly

TEXT

```
-rwxr-x---  1 deploy  app   4096 Mar 11 09:30 run.sh
|└┴┘└┴┘└┴┘  |   |      |
| |  |  |   |   |      |
| |  |  |   |   |      └── other: ---  (0)
| |  |  |   |   └──────── group: r-x  (5)
| |  |  |   └──────────── owner: rwx  (7)
| |  |  └──────────────── type: - file, d dir, l symlink, s socket, c char dev, b block dev, p fifo
```

Bit	On a file	On a directory
`r` (4)	Read contents	List names (but not stat them)
`w` (2)	Modify contents	Create/delete/rename entries
`x` (1)	Execute	Traverse — needed to access anything inside

> **The classic confusion**
>
> Deleting a file depends on the **directory's** write bit, not the file's. A read-only file in a writable directory can be removed by anyone with write access to that directory — which is exactly why `/tmp` carries the sticky bit.

Octal, symbolic, and umask

BASH

```
chmod 750 deploy.sh        # rwx r-x ---
chmod u+x,g-w,o= file      # symbolic form
chmod -R u+rwX,go+rX dir   # capital X = execute only where it already is, or on dirs
chown -R svc_app:app /opt/app
chgrp app /var/log/app

umask                      # 0022 typical → new files 644, new dirs 755
umask 0027                 # new files 640, dirs 750 - group-readable, world-blind
```

umask is subtractive from 666 for files and 777 for directories; it never grants execute to a regular file. Set it in the systemd unit (`UMask=0027`) rather than in a shell rc file, because services do not read your rc files.

setuid, setgid, sticky

Bit	Octal	Effect	Example
setuid	4000	Binary runs as its *owner*, not the caller	`/usr/bin/passwd` (`-rwsr-xr-x`)
setgid	2000	On a binary: runs as owning group. On a **directory**: new files inherit the directory's group	Shared team directory
sticky	1000	In a directory, only the file owner may delete	`/tmp` (`drwxrwxrwt`)

BASH

```bash
# Shared drop directory where everything lands in group "analytics"
sudo install -d -m 2770 -o root -g analytics /srv/dropbox
ls -ld /srv/dropbox      # drwxrws---

# Audit for unexpected setuid binaries (a real finding on a hardening review)
find / -xdev \( -perm -4000 -o -perm -2000 \) -type f -printf '%M %u %p\n' 2>/dev/null
```

Users, groups, and identity

BASH

```bash
id                                  # uid, gid, all supplementary groups
id -nG svc_app                      # group names for another user
getent passwd deploy                # works with LDAP/SSSD, unlike grep /etc/passwd
getent group docker
groups                              # your groups in the CURRENT session

sudo useradd -r -s /usr/sbin/nologin -d /opt/app -M svc_app   # service account
sudo usermod -aG docker "$USER"     # -a is mandatory, without it you REPLACE groups
newgrp docker                       # pick up the new group without logging out
```

✕ **`usermod -G` without `-a`**

This replaces all supplementary groups. Running `sudo usermod -G docker alice` can remove alice from `sudo`, `wheel` and every other group in one keystroke. Always `-aG`. Always check `id alice` before and after.

sudo: know your own blast radius

BASH

```bash
sudo -l                              # EXACTLY what you are permitted to run
sudo -u svc_app -- /opt/app/bin/health   # run as a service account
sudo -i                              # full login shell as root (loads root's env)
sudo -E env | grep -i proxy          # preserve YOUR environment (often blocked)
sudo visudo -f /etc/sudoers.d/deploy     # always edit via visudo - it validates
```

TEXT

```
# /etc/sudoers.d/deploy  - least privilege, not blanket ALL
Cmnd_Alias APPCTL = /bin/systemctl restart app, /bin/systemctl status app, \
                    /usr/bin/journalctl -u app *
%deploy ALL=(root) NOPASSWD: APPCTL
Defaults:%deploy  env_keep += "HTTP_PROXY HTTPS_PROXY NO_PROXY"
```

Note `env_reset` is the default: sudo strips your environment, which is why `sudo pip install` suddenly cannot reach the proxy. The `env_keep` line above is the fix, and it is a change you will request from a customer's platform team more than once.

POSIX ACLs when owner/group/other is not enough

BASH

```bash
getfacl /srv/shared
setfacl -m u:alice:rwx /srv/shared              # grant one user
setfacl -m g:auditors:rx /srv/shared            # grant one group
setfacl -d -m g:analytics:rwx /srv/shared       # DEFAULT acl: applies to new children
setfacl -x u:alice /srv/shared                  # remove
setfacl -b /srv/shared                          # strip all ACLs
```

A trailing `+` in `ls -l` (`drwxrwx---+`) means ACLs are present. If permissions look right but access still fails, check `getfacl` — and then check SELinux.

> ⚠ **RHEL: it is probably SELinux**
> On RHEL/CentOS/Rocky/Amazon Linux, "Permission denied" with perfect-looking permissions usually means an SELinux denial. Check `getenforce` , then `sudo ausearch -m avc -ts recent` or `sudo journalctl -t setroubleshoot` . Fix with the right context (`semanage fcontext -a -t httpd_sys_content_t '/srv/web(/.*)?' && restorecon -Rv /srv/web`) — never by telling the customer to run `setenforce 0` .

Processes, Signals, Job Control, and Inspection

"The service is up but requests hang." Answering that means reading process state, finding which process holds which port, seeing what syscall it is stuck in, and knowing which signal to send. These tools turn a vague complaint into a specific cause.

Seeing what runs

BASH

```
ps aux                                  # BSD syntax: everything, user-oriented
ps -ef                                  # System V syntax: same idea, different columns
ps -eo pid,ppid,user,%cpu,%mem,rss,etime,stat,cmd --sort=-%cpu | head -20
ps -eLf                                 # include threads
pgrep -af gunicorn                      # PIDs + full command line, pattern match
pstree -pa 1                            # process tree with PIDs and args
```

STAT	Meaning	FDE reading
`R`	Running or runnable	Burning CPU
`S`	Interruptible sleep	Normal — waiting on I/O or a timer
`D`	Uninterruptible sleep	Blocked on disk/NFS. Cannot be killed. Suspect storage
`Z`	Zombie	Exited, parent never reaped. Many zombies = buggy parent (common PID-1 problem in containers)
`T`	Stopped	Someone pressed `Ctrl`+`Z` and walked away
`<` / `N`	High / low priority	Someone niced it

top and htop in anger

TEXT

```
top - 14:22:11 up 42 days,  3:11,  2 users,  load average: 8.41, 6.02, 3.88
Tasks: 312 total,   3 running, 308 sleeping,   0 stopped,   1 zombie
%Cpu(s): 41.2 us,  9.8 sy,  0.0 ni, 12.4 id, 35.9 wa,  0.0 hi,  0.7 si
MiB Mem :  15894.0 total,    412.3 free,  13201.8 used,   2279.9 buff/cache
```

Read it top-down: load average 8.41 on (check `nproc`) 4 cores means a deep run queue. `35.9 wa` is iowait — the CPU is idle waiting on storage, so this is a disk problem, not a CPU problem. `412 MiB free` with low `buff/cache` means memory pressure is real and the OOM killer is close.

Inside `top`: `1` shows per-core, `M` sorts by memory, `P` by CPU, `c` shows full command lines, `H` toggles threads, `e` changes units. `htop` adds tree view (`F5`), filtering (`F4`) and per-process strace, but is not installed everywhere — know plain `top`.

Signals

Signal	No.	Default	When you send it
SIGHUP	1	Terminate	Reload config without restart (nginx, sshd, rsyslog)
SIGINT	2	Terminate	`Ctrl`+`C`
SIGQUIT	3	Core dump	Java: dumps all thread stacks to stdout — invaluable for hangs
SIGKILL	9	Die, uncatchable	Last resort. No cleanup, no flush, corrupt state possible
SIGTERM	15	Terminate	The polite default. `kill` with no flag sends this
SIGUSR1/2	10/12	Terminate	App-defined: log rotation, debug toggles
SIGSTOP/CONT	19/18	Pause/resume	Freeze a runaway process without losing its state

BASH

```bash
kill -TERM 4821          # by name, clearer than -15
kill -0 4821             # does the process exist / can I signal it? (no signal sent)
pkill -f 'gunicorn.*api' # pattern against the full command line
killall -u svc_app       # everything owned by a user
kill -QUIT $(pgrep -f java)   # thread dump to the JVM's stdout
```

Escalate, do not jump

Send SIGTERM, wait 10–30 s, then SIGKILL only if needed. A SIGKILL on Postgres means crash recovery on next start; on a Java app it means no shutdown hooks, no connection drain, and a burst of 502s at the load balancer.

Job control

BASH

```
long_task &              # start in background
jobs -l                  # list with PIDs
fg %1                    # bring job 1 forward
bg %1                    # resume a stopped job in the background
Ctrl+Z                   # suspend the foreground job (SIGTSTP)
disown -h %1             # detach from the shell so SIGHUP on logout won't kill it
nohup long_task > out.log 2>&1 &   # immune to HUP from the start
wait                     # block until all background jobs finish
wait -n                  # block until the NEXT one finishes (bash 4.3+)
```

Priority and limits

BASH

```
nice -n 10 ./reindex.sh              # start lower priority (-20 highest .. 19 lowest)
renice -n 5 -p 4821                  # change a running process
ionice -c3 -p 4821                   # idle I/O class - huge for backup jobs
ulimit -a                            # your limits
ulimit -n 65535                      # file descriptors for this shell
cat /proc/4821/limits                # the real, effective limits of a running process
```

"Too many open files"

The soft limit is often 1024. Raising it needs all three: `/etc/security/limits.conf` for login sessions, `LimitNOFILE=` in the systemd unit for services, and sometimes `fs.file-max` via sysctl. Verify with `cat /proc/<pid>/limits`, not with `ulimit -n` in your own shell — they are different contexts.

lsof: who has the file, who has the port

BASH

```
lsof -i :8080                # who is listening on / connected to 8080
lsof -i -P -n | grep LISTEN  # all listeners, numeric, no DNS lag
lsof -p 4821                 # everything this PID has open
lsof /var/log/app/app.log    # which processes hold this file
lsof +D /opt/app             # everything under a directory (slow)
lsof | grep deleted          # deleted-but-open files eating disk
lsof -u svc_app              # by user
```

That `grep deleted` line solves a real incident: `df` says the disk is 100% full, `du` says only 40% is used. A process is holding a deleted 60 GB log file open, so the space is not reclaimed until the process closes the descriptor or restarts.

strace and friends when nothing else explains it

BASH

```
strace -f -p 4821                          # attach, follow forks
strace -f -e trace=openat,stat -p 4821     # only file-lookup syscalls
strace -f -e trace=network -p 4821         # connect/send/recv
strace -c -p 4821                          # summary: which syscall dominates
strace -f -tt -T -o /tmp/trace.txt ./app   # timestamps + per-call duration
ltrace -p 4821                             # library calls instead of syscalls
```

Typical outcome: you attach to a hanging process and see it stuck on `connect(3, {sa_family=AF_INET, sin_port=htons(443), sin_addr=10.4.2.19}, 16` for sixty seconds — now you know it is a blocked egress to a specific IP, not an application bug.

> ⚠ **strace may be blocked**
> It needs `CAP_SYS_PTRACE`, and many hardened hosts set `kernel.yama.ptrace_scope=1` or higher. Check with `cat /proc/sys/kernel/yama/ptrace_scope`. If it is blocked, fall back to `/proc/<pid>/stack`, `/proc/<pid>/wchan`, `ss -tnp` and application-level debug logging.

systemd: Units, systemctl, journalctl, and Timers

On any mainstream distro since roughly 2015, systemd is PID 1. It starts services, supervises them, captures their logs, enforces resource limits, and replaces half of cron. If you are shipping software into a customer VM rather than a cluster, a well-written unit file is your deployment artifact.

The commands that matter

BASH

```
systemctl status app                # state + recent log lines + the exact ExecStart
systemctl start|stop|restart app
systemctl reload app                # if the unit defines ExecReload
systemctl enable --now app          # start now AND at boot
systemctl disable --now app
systemctl is-active app; systemctl is-enabled app     # script-friendly, exit codes
systemctl cat app                   # full unit file including drop-ins
systemctl edit app                  # create an override drop-in (the right way)
systemctl show app -p MainPID -p MemoryCurrent -p NRestarts
systemctl daemon-reload             # ALWAYS after editing unit files on disk
systemctl list-units --failed       # the first thing to run on a sick host
systemctl list-dependencies app
systemd-analyze blame               # slowest units at boot
```

Edit overrides, never the vendor file

Changing `/usr/lib/systemd/system/app.service` means the next package upgrade silently reverts you. Use `systemctl edit app`, which creates `/etc/systemd/system/app.service.d/override.conf`. That file wins, survives upgrades, and shows up clearly in `systemctl cat`.

A production-grade service unit

TEXT

```
# /etc/systemd/system/inference-api.service
[Unit]
Description=Inference API (FastAPI + uvicorn)
Documentation=https://wiki.internal/runbooks/inference-api
After=network-online.target postgresql.service
Wants=network-online.target
StartLimitIntervalSec=300
StartLimitBurst=5

[Service]
Type=exec
User=svc_api
Group=app
WorkingDirectory=/opt/inference-api/current
Environment="PYTHONUNBUFFERED=1"
EnvironmentFile=-/etc/inference-api/env        # leading - = tolerate absence
ExecStartPre=/opt/inference-api/current/bin/preflight.sh
ExecStart=/opt/inference-api/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 808
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5s
TimeoutStopSec=30s
KillSignal=SIGTERM
UMask=0027

# Resource ceilings - the customer's SRE team will ask about these
MemoryMax=6G
MemoryHigh=5G
CPUQuota=200%
LimitNOFILE=65535
TasksMax=512

# Hardening - cheap, and it makes security review go quickly
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/inference-api /var/log/inference-api
ProtectKernelTunables=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
SystemCallFilter=@system-service

[Install]
WantedBy=multi-user.target
```

Type=	systemd considers it started when…	Use for
`simple`	Immediately after fork (legacy default)	Rarely — prefer `exec`
`exec`	After `execve()` succeeds	Most modern services; catches "binary not found"
`forking`	Parent exits; needs `PIDFile=`	Old daemons that background themselves
`notify`	Service calls `sd_notify(READY=1)`	Accurate readiness ordering
`oneshot`	Process exits; pair with `RemainAfterExit=yes`	Migrations, setup tasks

journalctl queries you will actually type

BASH

```
journalctl -u inference-api -f                  # follow
journalctl -u inference-api --since '30 min ago'
journalctl -u inference-api --since '2026-03-11 09:00' --until '2026-03-11 09:30'
journalctl -u inference-api -p err              # priority err and worse (0-7)
journalctl -u inference-api -n 200 --no-pager
journalctl -u inference-api -o json-pretty | jq -r '.MESSAGE'
journalctl -b            # this boot;  -b -1  previous boot
journalctl -k            # kernel ring buffer (= dmesg)
journalctl _PID=4821 ; journalctl _UID=1001
journalctl --disk-usage ; sudo journalctl --vacuum-size=500M
journalctl -u inference-api --grep 'Timeout|5[0-9]{2}'   # regex, systemd 237+
```

⚠ **Volatile journals lose your evidence**

If `/var/log/journal` does not exist, the journal is memory-only and vanishes on reboot. Create it (`sudo mkdir -p /var/log/journal && sudo systemd-tmpfiles --create --prefix /var/log/journal`) or set `Storage=persistent` in `/etc/systemd/journald.conf` — otherwise the crash you are investigating left no trace.

Timers vs cron

TEXT

```
# /etc/systemd/system/reindex.service
[Unit]
Description=Nightly vector reindex
[Service]
Type=oneshot
User=svc_api
ExecStart=/opt/inference-api/bin/reindex.sh

# /etc/systemd/system/reindex.timer
[Unit]
Description=Run reindex nightly
[Timer]
OnCalendar=*-*-* 02:30:00
RandomizedDelaySec=600        # jitter so 400 VMs don't stampede the DB
Persistent=true               # run on boot if the machine was off at 02:30
AccuracySec=1min
Unit=reindex.service
[Install]
WantedBy=timers.target
```

BASH

```bash
sudo systemctl enable --now reindex.timer
systemctl list-timers --all           # NEXT, LEFT, LAST, PASSED — all in one view
systemd-analyze calendar 'Mon *-*-* 06:00:00'   # verify an OnCalendar expression
sudo systemctl start reindex.service   # test the job immediately, without waiting
```

Concern	cron	systemd timer
Logs	Mailed, or lost	In the journal, per-unit, queryable
Missed runs	Skipped silently	`Persistent=true` catches up
Overlap	Runs concurrently unless you add flock	Blocked by default (one instance per unit)
Environment	Minimal PATH, no profile	Explicit `Environment=` / `EnvironmentFile=`
Resource limits	None	Full cgroup controls
Availability	Everywhere, including containers	Needs systemd as PID 1

Cron still wins in containers and on minimal images. When you do use it, remember `%` must be escaped in crontabs, set `PATH=` and `SHELL=` at the top, redirect output explicitly (`>> /var/log/job.log 2>&1`), and wrap with `flock -n /var/lock/job.lock` to prevent overlap.

Package Management Across Distros and Air-Gapped Installs

You will not get to pick the distro. You will get a RHEL 8 box at one customer and an Alpine container at the next, and in both cases somebody will ask "can you just install it?" while the host has no route to the internet. Know the four package families and the offline workflow for each.

Identify first

BASH

```
cat /etc/os-release
# ID=rhel   VERSION_ID="8.9"        → dnf/yum, RPM
# ID=ubuntu VERSION_ID="22.04"      → apt, DEB
# ID=sles   VERSION_ID="15.5"       → zypper, RPM
# ID=alpine VERSION_ID="3.19"       → apk
# ID=amzn   VERSION_ID="2023"       → dnf, RPM

command -v apt dnf yum zypper apk pacman 2>/dev/null
```

The cross-distro command table

Task	apt (Debian/Ubuntu)	dnf / yum (RHEL family)	zypper (SUSE)	apk (Alpine)
Refresh metadata	`apt update`	`dnf makecache`	`zypper refresh`	`apk update`
Install	`apt install -y nginx`	`dnf install -y nginx`	`zypper -n in nginx`	`apk add nginx`
Remove	`apt remove` / `purge`	`dnf remove`	`zypper rm`	`apk del`
Search	`apt search` / `apt-cache search`	`dnf search`	`zypper se`	`apk search`
Info	`apt show nginx`	`dnf info nginx`	`zypper info`	`apk info -a`
What provides file X?	`dpkg -S /usr/sbin/nginx`	`dnf provides */nginx`	`zypper se --provides`	`apk info --who-owns`
List installed	`dpkg -l` / `apt list --installed`	`rpm -qa`	`rpm -qa`	`apk info`
Files in a package	`dpkg -L nginx`	`rpm -ql nginx`	`rpm -ql nginx`	`apk info -L`
Download only	`apt install --download-only`	`dnf download --resolve`	`zypper --pkg-cache-dir . in -d`	`apk fetch -R`
Install local file	`dpkg -i x.deb; apt -f install`	`dnf install ./x.rpm`	`zypper in ./x.rpm`	`apk add --allow-untrusted x.apk`
Hold version	`apt-mark hold nginx`	`dnf versionlock add`	`zypper al nginx`	pin in repo file
Upgrade all	`apt upgrade`	`dnf upgrade`	`zypper up`	`apk upgrade`

Repositories, mirrors, and the corporate proxy

BASH

```bash
# Debian/Ubuntu
cat /etc/apt/sources.list /etc/apt/sources.list.d/*.list
echo 'Acquire::http::Proxy "http://proxy.corp:3128";' | sudo tee /etc/apt/apt.conf.d/9

# RHEL family
ls /etc/yum.repos.d/
dnf repolist ; dnf config-manager --set-enabled powertools
sudo sed -i '/^\[main\]/a proxy=http://proxy.corp:3128' /etc/dnf/dnf.conf

# Point at the customer's internal mirror instead of upstream
sudo sed -i 's|http://archive.ubuntu.com|https://nexus.corp/repository/ubuntu|g' \
  /etc/apt/sources.list
```

> ⚠ **GPG and the corporate TLS-inspecting proxy**
> Many enterprises terminate TLS at the proxy with their own CA. Package tools then fail with certificate errors. The correct fix is to install the corporate CA into the system trust store — `sudo cp corp-root.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates` on Debian, or `sudo cp corp-root.crt /etc/pki/ca-trust/source/anchors/ && sudo update-ca-trust` on RHEL. Never disable verification as a workaround you leave behind.

Air-gapped install: the pattern

You have a connected staging machine of the *same distro, version and architecture*, and a target with no egress. Resolve dependencies on the connected box, carry the artifacts, install offline.

1. **Match the environment**
   Same `VERSION_ID` and `uname -m`. A container of the matching base image is the cleanest way: `docker run --rm -it -v "$PWD/out:/out" rockylinux:8 bash`.

2. **Download with dependencies**
   RHEL: `dnf download --resolve --alldeps --destdir /out nginx`. Debian: `apt-get install --download-only -o Dir::Cache::archives=/out nginx`. Alpine: `apk fetch -R -o /out nginx`.

3. **Build a local repo (RPM) or just copy (DEB)**
   `createrepo_c /out` produces repodata so `dnf` can resolve offline.

4 **Transfer**

`tar czf bundle.tgz out/`, then `sha256sum bundle.tgz` and record the hash in the change ticket. Move via the approved channel (SFTP drop, removable media with scan, artifact promotion).

5 **Install on target**

RPM: add a `baseurl=file:///srv/offline` repo file and `dnf --disablerepo='*' --enablerepo=offline install nginx`. DEB: `sudo dpkg -i *.deb || sudo apt-get -f install --no-download`.

6 **Verify**

`rpm -V nginx` / `debsums nginx`, then `nginx -v` and a smoke test. Record what you installed in the handover note.

BASH

```
# Language ecosystems have the same pattern
pip download -r requirements.txt -d wheels/ --platform manylinux2014_x86_64 \
    --python-version 311 --only-binary=:all:
pip install --no-index --find-links=wheels/ -r requirements.txt

npm ci --omit=dev && tar czf node_modules.tgz node_modules   # or use `npm pack`/Verda
# Containers: docker save / docker load
docker pull myreg/api:1.4.2 && docker save myreg/api:1.4.2 | gzip > api-1.4.2.tar.gz
gunzip -c api-1.4.2.tar.gz | docker load
```

✦ **Static binaries are your friend**

For single-purpose tools, a statically linked binary (Go, Rust, or a musl build) is one `scp` and a `chmod +x` away from working — no dependency resolution, no package approval. Keep a personal kit of `jq`, `yq`, `ripgrep`, `fd`, `kubectl`, `busybox` as static amd64 and arm64 binaries. Verify checksums against upstream and get them approved once, not every engagement.

SSH Deep Dive

SSH is the FDE's transport layer. It is how you reach the host, how you tunnel to a database behind a firewall, how you copy a 4 GB model file, and how you get a browser

onto an internal dashboard. Most engineers use ten percent of it. The other ninety percent is what gets you unblocked without filing a network ticket.

Keys

BASH

```bash
ssh-keygen -t ed25519 -C "pranit@fde-laptop" -f ~/.ssh/id_ed25519_acme
# ed25519: small, fast, modern. Use RSA 4096 only if the customer's tooling demands it
ssh-keygen -t rsa -b 4096 -C "..." -f ~/.ssh/id_rsa_legacy

ssh-copy-id -i ~/.ssh/id_ed25519_acme.pub deploy@bastion.acme.internal
# No ssh-copy-id? Do it by hand:
cat ~/.ssh/id_ed25519_acme.pub | ssh deploy@host \
  'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys'

ssh-keygen -lf ~/.ssh/id_ed25519_acme.pub      # fingerprint, to confirm with the cust
ssh-keygen -p -f ~/.ssh/id_ed25519_acme        # change the passphrase
```

Permissions kill SSH silently

`~/.ssh` must be `700`, private keys `600`, `authorized_keys` `600`, and your **home directory must not be group-writable**. StrictModes rejects anything looser and the client-side error is just "Permission denied (publickey)". Debug with `ssh -vvv` on the client and `sudo journalctl -u sshd -f` on the server.

~/.ssh/config — the highest-leverage file you own

TEXT

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 4
    TCPKeepAlive yes
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist 10m
    HashKnownHosts no
    AddKeysToAgent yes

Host bastion-acme
    HostName bastion.acme.example.com
    User pranit.deshpande
    Port 2222
    IdentityFile ~/.ssh/id_ed25519_acme
    IdentitiesOnly yes

# Every private host reached THROUGH the bastion, by one rule
Host 10.20.*.* app-* db-*
    User deploy
    IdentityFile ~/.ssh/id_ed25519_acme
    IdentitiesOnly yes
    ProxyJump bastion-acme

Host gpu-node
    HostName 10.20.4.17
    ProxyJump bastion-acme
    LocalForward 8888 127.0.0.1:8888      # Jupyter
    LocalForward 5432 10.20.9.4:5432      # Postgres behind the same network
    RequestTTY yes
```

`ControlMaster` multiplexes: the second and subsequent connections to the same host reuse the first TCP session and skip the handshake entirely — on a high-latency VPN that turns a 3-second connect into 50 ms. `IdentitiesOnly yes` stops the agent from offering every key you own, which matters when a customer's server has `MaxAuthTries 3` and locks you out after three wrong keys.

Agent and agent forwarding

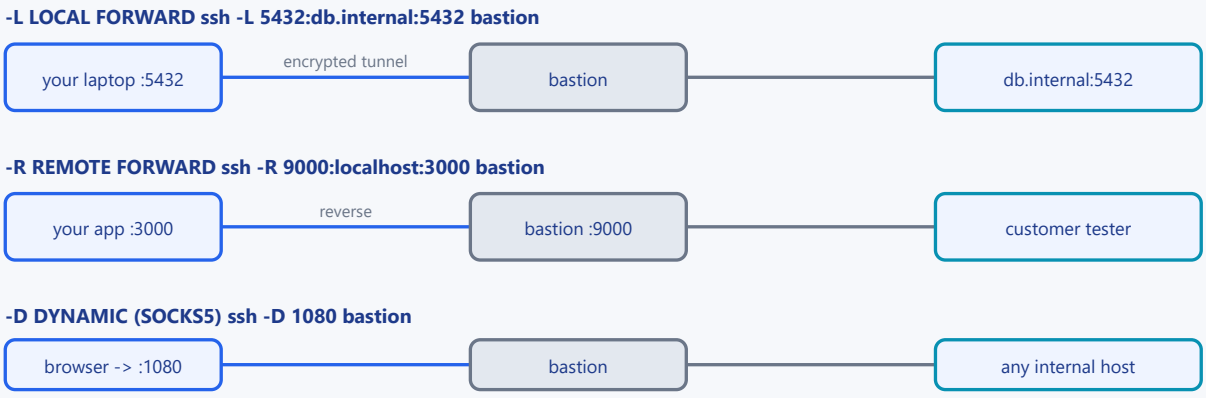
BASH

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_acme
ssh-add -l                      # loaded keys
ssh-add -t 3600 ~/.ssh/key      # auto-expire after an hour
ssh -A bastion                  # forward the agent
```

⚠ **Prefer ProxyJump to agent forwarding**

`-A` exposes your agent socket to anyone with root on the intermediate host; they can use your key for as long as you are connected. `ssh -J bastion target` (or `ProxyJump`) keeps the authentication on your laptop and only tunnels the TCP stream. Use `-A` only when you genuinely need to authenticate *from* the remote host (e.g. `git clone` on the server) and the customer permits it.

Port forwarding — the three directions



Local brings a remote service to you; remote exposes your service to them; dynamic turns SSH into a SOCKS proxy for everything.

BASH

```
# Local: reach a Postgres that only the bastion can see
ssh -N -L 5432:db.internal:5432 bastion-acme
psql -h 127.0.0.1 -p 5432 -U app appdb

# Multiple at once, in the background, no shell
ssh -f -N -L 8080:app-1:8080 -L 9090:prometheus:9090 -L 5432:db.internal:5432 bastion-

# Remote: let a customer tester hit the demo running on your laptop
ssh -N -R 9000:localhost:3000 bastion-acme
# (needs GatewayPorts yes on the server for non-loopback binding)

# Dynamic SOCKS5 proxy for a browser or curl
ssh -f -N -D 1080 bastion-acme
curl --socks5-hostname 127.0.0.1:1080 http://grafana.internal:3000/api/health

# Manage forwards on a live session: press Enter, then ~C for the ssh prompt
# ~. drops a hung session; ~? lists all escape sequences
```

Copying files: scp vs rsync

BASH

```
scp -C model.bin deploy@app-1:/opt/app/          # -C compresses; fine for one small
scp -r ./dist deploy@app-1:/opt/app/releases/
scp -3 host-a:/f host-b:/f                       # relay through your machine

# rsync is what you actually want for anything repeated or large
rsync -avz --partial --progress ./dist/ deploy@app-1:/opt/app/current/
rsync -avz --delete --exclude '.git' --exclude 'node_modules' ./src/ app-1:/opt/app/sr
rsync -avz -e 'ssh -J bastion-acme' ./dist/ app-1:/opt/app/     # through a jump host
rsync -avz --bwlimit=5000 bigfile host:/data/                   # 5 MB/s cap, be a goo
rsync -avzn --delete ./a/ host:/b/                              # -n = DRY RUN. Always
```

Trailing slashes in rsync

`rsync -a src/ dst/` copies the *contents* of src into dst. `rsync -a src dst/` creates `dst/src`. This single character has caused more misplaced deployments than any other rsync feature. And `--delete` plus a wrong path is how you erase a customer's directory — run with `-n` first, every time.

Server-side hardening you may be asked to review

TEXT

```
# /etc/ssh/sshd_config.d/99-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowGroups ssh-users
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowTcpForwarding yes          # FDEs need this; justify it rather than lose it
LogLevel VERBOSE
```

Validate before restarting — `sudo sshd -t` — and keep your existing session open while you restart, so a mistake does not lock you out.

Text Processing: grep, sed, awk, cut, sort, jq

Logs, configs and API responses are all text. The ability to go from a 2 GB log file to "these 7 request IDs failed, all from one upstream" in a single pipeline is the most visible skill you have in front of a customer.

grep and the regex you need

BASH

```
grep -r 'apiKey' /opt/app --include='*.py'     # recursive, filtered by glob
grep -ri 'timeout' -n /etc/nginx               # case-insensitive, line numbers
grep -c ERROR app.log                          # count matching LINES
grep -v DEBUG app.log                          # invert
grep -w id app.log                             # whole word - not "uuid", not "identit
grep -A3 -B3 'Traceback' app.log               # context after / before
grep -C5 'OOM' /var/log/syslog                 # 5 lines either side
grep -l 'LoadBalancer' *.yaml                  # just the filenames
grep -o 'req_id=[a-f0-9]\{12\}' app.log        # print only the match
grep -E '5[0-9]{2} ' access.log                # ERE: no backslashes needed
grep -F '$special[chars]' file                 # fixed string, no regex
grep -P '(?<=user=)\w+' app.log                # PCRE lookbehind (GNU only)
```

Pattern	Matches	Pattern	Matches
`^` `$`	Line start / end	`\b`	Word boundary
`.`	Any single char	`[^abc]`	Any char except a, b, c
`*` `+` `?`	0+, 1+, 0 or 1	`{2,5}`	Between 2 and 5 repeats
`(a|b)`	Alternation (ERE)	`\d \w \s`	Digit, word char, whitespace (PCRE)

ripgrep when it exists

`rg 'pattern' --type py -C3` is 5–20× faster than `grep -r`, respects `.gitignore`, and defaults to recursive. But it is not installed on customer boxes, so keep GNU grep in your fingers and treat `rg` as a local luxury.

sed: stream edits

BASH

```
sed 's/localhost/db.internal/' config.ini          # first occurrence per line
sed 's/old/new/g' config.ini                       # all occurrences
sed -i.bak 's/DEBUG/INFO/g' app.conf               # in place, keeping app.conf.bak
sed -n '120,140p' huge.log                         # print a line range only
sed -n '/BEGIN/,/END/p' dump.txt                   # range between two patterns
sed '/^#/d; /^$/d' nginx.conf                      # strip comments and blank lines
sed '3i\Inserted before line 3' file
sed 's|/old/path|/new/path|g' file                 # alternate delimiter for paths
sed -E 's/([0-9]{4})-([0-9]{2})/\2\/\1/' dates.txt # ERE with capture groups
```

> ✕ **`sed -i` is not portable and is not reversible**
>
> GNU sed accepts `-i` with an optional suffix; BSD/macOS sed *requires* an argument (`sed -i '' 's/a/b/'`). On a customer host, always use a suffix (`-i.bak`) so you have a rollback, and preview without `-i` first. One mistyped regex across `/etc` is an outage.

awk: the pocket language

BASH

```
awk '{print $1, $7}' access.log                    # fields; $0 is the whole line, NF
awk -F: '{print $1}' /etc/passwd                   # custom separator
awk -F'\t' 'NR>1 {print $3}' data.tsv              # skip header
awk '$9 ≥ 500 {print $7}' access.log               # filter then project
awk '{s+=$10} END {print s/1048576 " MB"}' access.log   # sum bytes column
awk '{c[$9]++} END {for (k in c) print c[k], k}' access.log | sort -rn   # status hist
awk '{t[$7]+=$NF; n[$7]++} END {for (u in t) printf "%8.3f %5d %s\n", t[u]/n[u], n[u],
    timings.log | sort -rn | head
awk 'length($0) > 200' app.log                     # abnormally long lines
awk '/BEGIN_TX/,/END_TX/' app.log                  # range patterns
awk 'BEGIN{OFS=","} {print $1,$4,$9}' access.log   # retabulate as CSV
```

That status-histogram one-liner is the single most useful awk you will write: it turns an opaque access log into "487 × 200, 61 × 499, 12 × 502" in under a second, and it works on any log format once you know the column index.

cut, sort, uniq, tr, column, and the counting idiom

BASH

```
cut -d, -f1,3 data.csv            # by delimiter and field
cut -c1-19 app.log                # by character position - great for timestamps
sort -u list.txt                  # sort + dedupe
sort -t, -k3,3nr -k1,1 data.csv   # 3rd field numeric descending, then 1st ascending
sort -h                           # human sizes: 1K 23M 4G
uniq -c                           # count runs - REQUIRES sorted input
uniq -d                           # only duplicates
tr -d '\r' < win.txt > unix.txt   # strip CRLF
tr '[:upper:]' '[:lower:]'
tr -s ' '                         # squeeze repeated spaces
paste -d, a.txt b.txt             # join files side by side
comm -13 old.txt new.txt          # lines only in new (both must be sorted)
join -t, -1 1 -2 1 a.csv b.csv    # relational join on a key
column -t -s, data.csv            # align into readable columns
nl -ba file                       # number every line
```

The universal frequency pipeline

`… | sort | uniq -c | sort -rn | head -20`. Top IPs hitting your API, most common error strings, busiest endpoints, repeated stack frames — all the same four stages. Commit it to muscle memory.

jq: JSON is now the default log format

BASH

```
curl -s https://api.internal/v1/pods | jq .
jq -r '.items[].metadata.name' pods.json             # -r drops the quotes
jq '.items[] | select(.status.phase ≠ "Running")' pods.json
jq -r '.items[] | [.metadata.name, .status.phase] | @tsv' pods.json | column -t
jq 'map(.latency_ms) | add/length' timings.json      # mean
jq -s 'group_by(.level) | map({level: .[0].level, n: length})' app.jsonl
jq -c '.' pretty.json                                # compact, one line
jq -e '.status == "ok"' resp.json >/dev/null         # EXIT CODE for scripts
jq --arg env prod '.envs[$env]' config.json          # inject a shell variable safely
jq -r 'to_entries[] | "\(.key)=\(.value)"' env.json  # JSON object → KEY=VALUE
cat app.jsonl | jq -r 'select(.level=="error") | "\(.ts) \(.req_id) \(.msg)"'
```

For YAML, `yq` (the Go rewrite by mikefarah) uses the same expression language: `yq '.spec.template.spec.containers[0].image' deploy.yaml`. If neither is installed and you have Python, `python3 -c 'import sys,json;d=json.load(sys.stdin);print(d["items"][0]["metadata"]["name"])'` always works.

A worked investigation

BASH

```bash
# "The API got slow around 14:00." Access log is combined format, $NF = upstream time.
awk '$4 ~ /11\/Mar\/2026:14:/' access.log \
  | awk '{print $7, $NF}' \
  | sort -k2 -rn | head -20
# → /v1/search dominates with 8–12 s responses

# Which upstream?  Count status codes for just that endpoint
grep ' /v1/search' access.log | awk '{print $9}' | sort | uniq -c | sort -rn
#  412 200
#   97 499     ← client gave up: timeouts, not errors

# Correlate with the app's own structured log
jq -r 'select(.path=="/v1/search" and .duration_ms>5000)
       | "\(.ts) \(.req_id) \(.db_ms) \(.vector_ms)"' app.jsonl | head
# → vector_ms is 7800 of the 8200 ms.  The vector store is the bottleneck.
```

Pipes, Redirection, File Descriptors, and Here-Docs

Every mysterious "my script logs nothing" or "the error disappeared" bug comes down to misunderstanding file descriptors. Three numbers, one ordering rule, and a handful of operators cover the whole topic.

FD	Name	Default
0	stdin	Keyboard / previous command in a pipe
1	stdout	Terminal — normal output
2	stderr	Terminal — errors and diagnostics, unbuffered

BASH

```
cmd > out.txt            # stdout, truncate
cmd >> out.txt           # stdout, append
cmd 2> err.txt           # stderr only
cmd > out.txt 2>&1       # both to out.txt   (ORDER MATTERS)
cmd &> out.txt           # bash shorthand for the same
cmd 2>&1 > out.txt       # WRONG: stderr goes to the TERMINAL, stdout to the file
cmd > /dev/null 2>&1     # silence everything
cmd 2>/dev/null          # keep output, drop the permission-denied noise
cmd < input.txt          # stdin from a file
cmd <<< "inline string"  # here-string
```

The ordering rule: redirections are applied left to right, and `2>&1` means "make fd 2 a copy of wherever fd 1 points *right now*". So you must redirect stdout first, then duplicate.

BASH

```
# Pipe stderr instead of stdout
cmd 2>&1 >/dev/null | grep -i error

# Both to the terminal AND a file
cmd 2>&1 | tee deploy.log
cmd 2>&1 | tee -a deploy.log      # append

# Capture stdout and stderr to separate files while still seeing both
cmd > >(tee out.log) 2> >(tee err.log >&2)

# Custom descriptors (useful when you need a side channel in a script)
exec 3> /var/log/audit-trail.log
echo "starting deploy $(date -Is)" >&3
exec 3>&-                         # close it

# Pipeline exit status
false | true; echo $?             # 0 - you got the LAST command's status
set -o pipefail; false | true; echo $?   # 1 - what you actually wanted
echo "${PIPESTATUS[@]}"           # per-stage statuses: "1 0"
```

Here-docs and here-strings

BASH

```
# Variables EXPAND
cat > /etc/app/config.yaml <<EOF
environment: ${ENV}
endpoint: https://${HOST}:8443
EOF

# Quoted delimiter: NOTHING expands - essential for scripts containing $ and `
cat > /usr/local/bin/wrapper.sh <<'EOF'
#!/usr/bin/env bash
exec /opt/app/bin/app --config "$1" "${@:2}"
EOF

# <<- strips LEADING TABS so you can indent inside a function
deploy() {
        ssh app-1 bash -s <<-'EOF'
                set -euo pipefail
                systemctl --user restart api
                systemctl --user is-active api
        EOF
}

# Here-string into a command expecting stdin
jq -r .name <<< "$json_response"
```

Process substitution

BASH

```
diff <(ssh app-1 'sha256sum /opt/app/config.yaml') \
     <(ssh app-2 'sha256sum /opt/app/config.yaml')

diff <(sort a.txt) <(sort b.txt)
comm -3 <(rpm -qa | sort) <(ssh other 'rpm -qa' | sort)   # package drift between host

# Feed a command's output to something that only accepts a FILE path
grep -f <(cut -d, -f1 ids.csv) app.log

# Output process substitution: tee into two different pipelines
./build.sh > >(gzip > build.log.gz) 2>&1
```

⚠ **The subshell variable trap**

`cat f | while read l; do n=$((n+1)); done; echo $n` prints nothing useful, because the right side of a pipe runs in a subshell and its variable changes are discarded. Use `while read l; do …; done < f` or `done < <(cmd)` instead. This is one of the top three bash bugs in production scripts.

Bash Scripting That Does Not Break

The script you write on a customer engagement outlives you. It will be run by someone else, on a different host, with a path containing a space, when the network is flaky. Defensive bash is a small set of habits that eliminate entire bug classes.

The preamble

BASH

```bash
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Flag	Effect	Gotcha
`-e`	Exit on any non-zero status	Not applied inside `if`, `&&`, `||` or `!` conditions — by design
`-u`	Error on unset variable	Use `"${VAR:-default}"` for optional vars; `"${arr[@]:-}"` for possibly-empty arrays
`-o pipefail`	Pipeline fails if any stage fails	Makes `cmd | head` fail with SIGPIPE; guard with `|| true` where intended
`-x`	Trace every command	Add temporarily, or gate on `${DEBUG:-}`. Will leak secrets into logs

`#!/usr/bin/env bash` rather than `#!/bin/bash` finds bash wherever it lives (`/usr/local/bin/bash` on BSD, Nix paths, Homebrew on macOS). And never write `#!/bin/sh` if you use arrays, `[[ ]]`, or `local` — on Debian that is dash and it will fail at runtime, not parse time.

Quoting: the rule

BASH

```bash
rm $file          # BROKEN: word-splits on spaces, then globs
rm "$file"        # correct
cp "$src" "$dst"
for f in "${files[@]}"; do …; done    # quoted [@] preserves element boundaries
echo "${arr[*]}"  # single string joined by the first char of IFS - usually NOT what y

[[ -z "${VAR:-}" ]] && echo "empty or unset"
[[ "$a" == "$b" ]]          # string equality (no word splitting inside [[ ]])
[[ "$s" == pre* ]]          # glob match - do NOT quote the right side
[[ "$s" =~ ^v[0-9]+\.[0-9]+$ ]]   # regex; captures land in ${BASH_REMATCH[@]}
(( count > 10 ))            # arithmetic context - no $ needed on variables
```

Run shellcheck on everything

`shellcheck deploy.sh` catches unquoted expansions, useless `cat`, subshell variable loss, and dozens of other real bugs. Wire it into CI. If you cannot install it on the customer box, run it locally before you ship the script — it is the single highest-value tool in bash development.

Arrays, parameter expansion, and argument parsing

BASH

```bash
args=(--verbose --config /etc/app.yaml)
args+=(--dry-run)
"${#args[@]}"                    # length
"${args[@]:1:2}"                 # slice
mapfile -t lines < file.txt      # read a file into an array, no splitting bugs
readarray -t pods < <(kubectl get po -o name)

"${VAR:-fallback}"       # use fallback if unset or empty
"${VAR:?must be set}"    # abort with a message if unset - great for required config
"${VAR:+--flag $VAR}"    # emit only if set
"${PATH//:/$'\n'}"       # global substitution
"${file%.tar.gz}"        # strip shortest suffix match
"${file##*/}"            # basename
"${file%/*}"             # dirname
"${name^^}" "${name,,}"  # upper / lower (bash 4+)
```

Traps and cleanup

BASH

```
tmp="$(mktemp -d)"
cleanup() {
  local rc=$?
  rm -rf "$tmp"
  [[ -n "${tunnel_pid:-}" ]] && kill "$tunnel_pid" 2>/dev/null || true
  (( rc ≠ 0 )) && echo "FAILED at line $LINENO (rc=$rc)" >&2
  return $rc
}
trap cleanup EXIT              # runs on normal exit AND on error with set -e
trap 'echo interrupted; exit 130' INT TERM
trap 'echo "error on line $LINENO: $BASH_COMMAND" >&2' ERR
```

A production-grade script

BASH

```
#!/usr/bin/env bash
#
# deploy.sh - release an artifact to an app host via an atomic symlink swap.
# Usage: deploy.sh -e prod -v 1.4.2 [-H app-1,app-2] [-n] [-h]
#
set -euo pipefail

readonly SCRIPT_NAME="${0##*/}"
readonly RELEASE_ROOT="/opt/inference-api/releases"
readonly CURRENT_LINK="/opt/inference-api/current"
readonly ARTIFACT_REPO="https://nexus.corp/repository/releases"

ENVIRONMENT="" ; VERSION="" ; DRY_RUN=0
HOSTS="app-1,app-2"
LOG_PREFIX="[$SCRIPT_NAME]"

log()  { printf '%s %s %s\n' "$(date -Is)" "$LOG_PREFIX" "$*" >&2; }
die()  { log "ERROR: $*"; exit 1; }
run()  { if (( DRY_RUN )); then log "DRY-RUN: $*"; else log "+ $*"; "$@"; fi; }

usage() {
  sed -n '2,6p' "$0" | sed 's/^# \{0,1\}//'
  exit "${1:-0}"
}

parse_args() {
  while getopts ':e:v:H:nh' opt; do
    case "$opt" in
      e) ENVIRONMENT="$OPTARG" ;;
      v) VERSION="$OPTARG" ;;
      H) HOSTS="$OPTARG" ;;
      n) DRY_RUN=1 ;;
      h) usage 0 ;;
      :) die "option -$OPTARG requires an argument" ;;
      \?) die "unknown option -$OPTARG (try -h)" ;;
    esac
  done
  [[ -n "$ENVIRONMENT" ]] || die "missing -e <environment>"
  [[ -n "$VERSION" ]]     || die "missing -v <version>"
  [[ "$VERSION" =~ ^[0-9]+\.[0-9]+\.[0-9]+$ ]] || die "version must be semver, got '$V
  [[ "$ENVIRONMENT" =~ ^(dev|staging|prod)$ ]] || die "env must be dev|staging|prod"
}

require_tools() {
  local missing=()
  for t in curl sha256sum ssh tar; do
    command -v "$t" >/dev/null 2>&1 || missing+=("$t")
  done
```

```
  (( ${#missing[@]} == 0 )) || die "missing required tools: ${missing[*]}"
}

WORKDIR=""
cleanup() {
  local rc=$?
  [[ -n "$WORKDIR" && -d "$WORKDIR" ]] && rm -rf "$WORKDIR"
  (( rc == 0 )) && log "done" || log "aborted with status $rc"
  exit $rc
}
trap cleanup EXIT
trap 'die "interrupted"' INT TERM

fetch_artifact() {
  local url="$ARTIFACT_REPO/api-${VERSION}.tar.gz"
  WORKDIR="$(mktemp -d -t deploy.XXXXXX)"
  log "fetching $url"
  curl -fsSL --retry 3 --retry-delay 2 --max-time 300 \
       -o "$WORKDIR/api.tar.gz" "$url" \
    || die "download failed - check HTTPS_PROXY and that $VERSION was published"
  curl -fsSL "$url.sha256" -o "$WORKDIR/api.tar.gz.sha256" || die "no checksum publish
  ( cd "$WORKDIR" && sha256sum -c api.tar.gz.sha256 ) || die "checksum mismatch - refu
  log "artifact verified ($(stat -c%s "$WORKDIR/api.tar.gz") bytes)"
}

deploy_host() {
  local host="$1" target="$RELEASE_ROOT/$VERSION"
  log "==> $host <=="
  run scp -q "$WORKDIR/api.tar.gz" "$host:/tmp/api-$VERSION.tar.gz"
  run ssh "$host" "mkdir -p '$target' && tar xzf /tmp/api-$VERSION.tar.gz -C '$target'
  run ssh "$host" "ln -sfn '$target' '$CURRENT_LINK'"
  run ssh "$host" "sudo systemctl restart inference-api"

  local attempt
  for attempt in 1 2 3 4 5 6 7 8 9 10; do
    if (( DRY_RUN )); then log "DRY-RUN: health check"; return 0; fi
    if ssh "$host" "curl -fsS --max-time 3 http://127.0.0.1:8080/healthz" >/dev/null 2
      log "$host healthy after ${attempt}s"; return 0
    fi
    sleep 1
  done
  log "$host FAILED health check - rolling back"
  local previous
  previous="$(ssh "$host" "ls -1dt '$RELEASE_ROOT'/*/ | sed -n 2p")"
  [[ -n "$previous" ]] || die "no previous release on $host - manual intervention need
  ssh "$host" "ln -sfn '${previous%/}' '$CURRENT_LINK' && sudo systemctl restart infer
  die "rolled $host back to ${previous%/}"
}
```

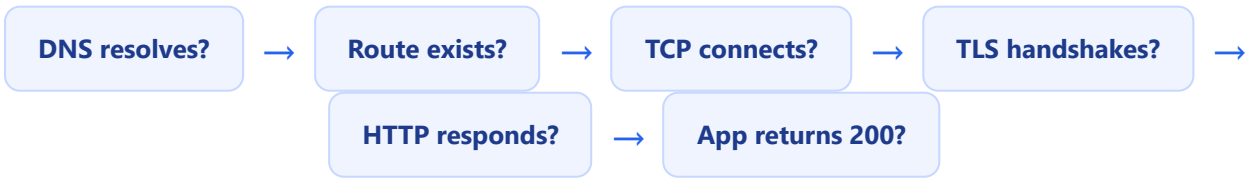
```
main() {
  parse_args "$@"
  require_tools
  log "deploying api $VERSION to $ENVIRONMENT (hosts: $HOSTS, dry-run=$DRY_RUN)"
  fetch_artifact
  local IFS=','
  for host in $HOSTS; do
    deploy_host "$host"
  done
  log "all hosts updated to $VERSION"
}

main "$@"
```

- ☐ `set -euo pipefail` and a trap-based cleanup that always runs
- ☐ Every variable quoted; validation of every user-supplied input before use
- ☐ `--dry-run` so the customer can see exactly what will happen first
- ☐ Checksum verification before anything is written to a target host
- ☐ Health check with bounded retries and an automatic rollback path
- ☐ Timestamped logs on stderr so stdout stays clean for piping
- ☐ Meaningful exit codes: 0 success, non-zero with a message explaining what to do

Networking From the Shell

"It can't reach the API" is the single most common sentence of an FDE engagement. Resolve it methodically — layer by layer — and you will find the answer in four commands instead of four hours of guessing.



Layer 1–3: interfaces, addresses, routes

BASH

```
ip -brief addr                  # compact: iface, state, addresses
ip addr show eth0
ip -brief link                  # UP/DOWN and MAC
ip route                        # routing table; "default via ..." is your gateway
ip route get 10.20.9.4          # WHICH route and source IP will be used for this dest
ip neigh                        # ARP cache
ip -s link show eth0            # RX/TX errors and drops
```

`ip route get` is underrated: it answers "will this traffic go out the VPN interface or the default one" definitively, which matters a lot in split-tunnel customer networks.

DNS

BASH

```
cat /etc/resolv.conf                        # nameservers and search domains
dig api.internal +short
dig api.internal @10.20.0.2 +short          # bypass the configured resolver
dig api.internal ANY +noall +answer
dig -x 10.20.9.4 +short                     # reverse lookup
getent hosts api.internal                   # uses NSSWITCH - includes /etc/hosts
resolvectl status                           # systemd-resolved: per-link DNS, DNSSEC st
host api.internal ; nslookup api.internal   # when dig is absent
```

> ⚠ **`dig` and `getent` can disagree**
>
> `dig` talks straight to a DNS server. `getent hosts` (and therefore your application) goes through NSS: `/etc/hosts` first, then mDNS, then DNS, per `/etc/nsswitch.conf`. If `dig` works but the app does not, check `/etc/hosts` for a stale override and check `search` domains — `api` may be resolving as `api.corp.local`.

Layer 4: sockets and connectivity

BASH

```
ss -tulpn                       # TCP+UDP listeners with process names (needs root for
ss -tn state established         # current connections
ss -tn 'dport = :5432'           # filter by destination port
ss -s                            # summary counts - spot socket exhaustion
ss -ti                           # per-socket TCP info: rtt, cwnd, retransmits

nc -zv db.internal 5432          # is the port open? (-z scan, -v verbose)
nc -zv -w3 api.internal 443      # with a 3 s timeout
timeout 3 bash -c '>/dev/tcp/db.internal/5432' && echo open || echo closed   # NO TOOL

traceroute -T -p 443 api.internal    # TCP traceroute - ICMP is usually blocked
mtr -rwc 20 api.internal             # continuous traceroute with loss stats
ping -c4 10.20.0.1                   # often blocked entirely in enterprise networks
```

That `/dev/tcp` trick is a bash built-in and works on hosts where `nc`, `telnet` and `curl` have all been removed by the hardening baseline. It is the one connectivity test you can always perform.

Layer 7: curl like you mean it

BASH

```
curl -v https://api.internal/v1/health           # headers + TLS details
curl -sS -o /dev/null -w '%{http_code} dns=%{time_namelookup}s tcp=%{time_connect}s \
tls=%{time_appconnect}s ttfb=%{time_starttransfer}s total=%{time_total}s\n' \
  https://api.internal/v1/health
curl -fsSL -H 'Authorization: Bearer '"$TOKEN" https://api/v1/items | jq .
curl -X POST -H 'Content-Type: application/json' -d @body.json https://api/v1/items
curl --resolve api.internal:443:10.20.9.4 https://api.internal/   # bypass DNS, keep S
curl -k https://api.internal/                     # skip TLS verify - DIAGNOSIS ONLY
curl --cacert /etc/ssl/corp-root.pem https://api.internal/
curl -x http://proxy.corp:3128 https://api.internal/
curl --retry 3 --retry-connrefused --max-time 20 …
curl -I https://api.internal/                     # HEAD only
```

The `-w` timing breakdown is the fastest way to localise slowness: a large `time_namelookup` is DNS, a large `time_connect` is network or firewall backlog, a large `time_appconnect` is TLS (often an OCSP or CRL fetch timing out through a proxy), and a large `time_starttransfer` with everything else fast is the application itself.

TLS inspection

BASH

```
openssl s_client -connect api.internal:443 -servername api.internal < /dev/null
openssl s_client -connect api.internal:443 -showcerts < /dev/null \
  | openssl x509 -noout -subject -issuer -dates
echo | openssl s_client -connect api.internal:443 2>/dev/null \
  | openssl x509 -noout -enddate            # expiry — check this before blaming code
openssl x509 -in /etc/ssl/corp-root.pem -noout -text | head -20
openssl verify -CAfile /etc/ssl/corp-root.pem server.pem
```

"x509: certificate signed by unknown authority"

The customer's TLS-inspecting proxy is re-signing traffic with a corporate root your tool does not trust. Fix it properly per runtime: system store via `update-ca-certificates` / `update-ca-trust`; Python via `REQUESTS_CA_BUNDLE` / `SSL_CERT_FILE` or `pip config set global.cert`; Node via `NODE_EXTRA_CA_CERTS`; Java via `keytool -importcert` into the cacerts truststore; Go via the system store. Disabling verification is never the answer you leave in place.

Proxies

BASH

```
export HTTP_PROXY=http://proxy.corp:3128
export HTTPS_PROXY=http://proxy.corp:3128      # yes, http:// for the proxy itself
export NO_PROXY=localhost,127.0.0.1,.internal,.svc,.cluster.local,10.0.0.0/8,169.254.1
export http_proxy="$HTTP_PROXY" https_proxy="$HTTPS_PROXY" no_proxy="$NO_PROXY"
```

Symptom	Likely proxy cause
Internal service calls hang for 30 s then fail	Missing from `NO_PROXY` — traffic is being sent to the proxy, which cannot route internally
Works in your shell, fails under systemd	Units do not read your profile; add `Environment=` or an `EnvironmentFile=`
Works for you, fails under `sudo`	`env_reset`; use `sudo -E` or add `env_keep` in sudoers
AWS/Azure metadata calls fail	`169.254.169.254` must be in `NO_PROXY`
Kubernetes in-cluster calls fail	`.svc`, `.cluster.local` and the pod/service CIDRs must be in `NO_PROXY`
Some tools honour it, others do not	Case sensitivity: Go reads both, some libraries only read lowercase. Set both

/etc/hosts and packet capture

BASH

```bash
# Temporary override to test against a specific backend before DNS is updated
echo '10.20.9.44  api.internal' | sudo tee -a /etc/hosts
getent hosts api.internal        # confirm it took effect

# tcpdump: when you need proof about what left the box
sudo tcpdump -i any -nn port 5432 -c 50
sudo tcpdump -i any -nn host 10.20.9.4 and port 443 -w /tmp/cap.pcap
sudo tcpdump -i any -nn -A 'tcp port 8080 and (((ip[2:2]-((ip[0]&0xf)<<2))-((tcp[12]&0
```

Reading a capture: `SYN` with no `SYN-ACK` means a firewall is dropping (silently) or the service is not listening; `SYN` followed by `RST` means something actively refused — usually nothing bound to that port or a firewall configured to reject rather than drop. That distinction tells the customer's network team exactly where to look.

> **Always clean up /etc/hosts**
> A forgotten hosts entry is a time bomb that fires weeks later when the real IP changes. Note every edit you make to a customer system in your handover document, and remove temporary ones before you leave.

Performance and Resource Triage

Something is slow or something stopped. You have access, no dashboards, and a customer on a call. These commands, in this order, localise the problem to CPU, memory, disk or network within two minutes.

Load average, correctly understood

BASH

```
uptime
#  14:22:11 up 42 days,  3:11,  2 users,  load average: 8.41, 6.02, 3.88
nproc              # 4  → load of 8.41 on 4 cores is a deep run queue
cat /proc/pressure/cpu /proc/pressure/io /proc/pressure/memory   # PSI, kernel 4.20+
```

Load average on Linux counts processes in **R** (runnable) *and* **D** (uninterruptible I/O wait). That is why a host with idle CPUs can show a load of 30 — the queue is blocked on a failing NFS mount, not on compute. The three numbers are 1-, 5- and 15-minute averages: rising means the problem is getting worse, falling means it peaked already. PSI (`/proc/pressure/*`) is far more precise when available — it reports the percentage of time tasks were stalled on each resource.

CPU

BASH

```
vmstat 1 5
# procs ———————memory———————— ---swap-- ————io——— -system-- ——————cpu——————
#  r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
#  9  2      0 421364  18244 2302120    0    0  1240  3300 8211 15002 41  9 15 35  0

mpstat -P ALL 1 3            # per-core (sysstat package)
pidstat -u 1 5               # per-process CPU over time
top -b -n1 -o %CPU | head -15   # batch mode - scriptable snapshot
```

Read the `r` column (runnable) against core count, `b` (blocked) for I/O trouble, `cs` (context switches) for thrashing, and `st` (steal) — non-zero steal on a cloud VM means the hypervisor is giving your cycles to someone else, which is a capacity conversation with the customer's cloud team, not a code problem.

Memory and the OOM killer

BASH

```
free -h
#               total   used   free  shared  buff/cache   available
# Mem:           15Gi   12Gi  412Mi    64Mi       2.2Gi      1.1Gi
# Swap:         2.0Gi  1.8Gi  200Mi
```

Ignore `free`; read **available**. Linux deliberately uses spare RAM for page cache and will evict it on demand. Low `available` with high swap usage means real pressure.

BASH

```
ps -eo pid,user,rss,vsz,comm --sort=-rss | head -10
smem -rs rss 2>/dev/null || true          # PSS accounting, better for shared memory
cat /proc/meminfo | grep -E 'MemAvailable|Dirty|Writeback|Slab'

# Did the OOM killer fire?
dmesg -T | grep -i -E 'out of memory|killed process|oom'
journalctl -k --since -24h | grep -i oom
grep -i 'killed process' /var/log/messages /var/log/syslog 2>/dev/null

# Per-process OOM tuning
cat /proc/4821/oom_score /proc/4821/oom_score_adj
echo -500 | sudo tee /proc/4821/oom_score_adj     # make it less likely to be chosen
```

TEXT

```
[Mar11 02:14] Out of memory: Killed process 4821 (python3) total-vm:9843212kB,
              anon-rss:7120448kB, file-rss:0kB, shmem-rss:0kB, UID:1001 pgtables:15104
```

That kernel line is conclusive: the process was not buggy in a way that produced a stack trace — the kernel shot it. `anon-rss:7120448kB` tells you it had reached 7.1 GB. Next step is either raising `MemoryMax`, adding RAM, or fixing the leak — but you now know *which* conversation to have.

Disk

BASH

```
df -h                       # space
df -i                       # INODES - a separate exhaustion mode entirely
du -xh --max-depth=1 /var 2>/dev/null | sort -h | tail
iostat -xz 1 5              # per-device: %util, await, r/s, w/s
iotop -oPa                  # per-process I/O (needs root)
lsblk -f                    # devices, filesystems, mountpoints, UUIDs
mount | column -t
findmnt -D                  # mounts with space, tree form
```

In `iostat -xz`, `%util` near 100 with high `await` (ms per I/O) means the device is saturated. On SSD/NVMe, `await` above ~10 ms is suspicious; on network-attached storage (EBS, NFS) it routinely means you hit a throughput or IOPS limit the customer provisioned.

Runbook: "disk full at 2am"

1. **Confirm which filesystem**
   `df -h | sort -k5 -rn | head`. Also run `df -i` — millions of tiny session files can exhaust inodes while space looks fine, and the error message is identical.

2. **Find the weight**
   `sudo du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -20`. Start at the mountpoint that is full; `-x` prevents wandering onto other filesystems.

3. **Check deleted-but-open files**
   `sudo lsof +L1` or `sudo lsof | grep deleted`. If `du` and `df` disagree by gigabytes, this is why. Restarting the holding process reclaims the space instantly.

4. **The usual suspects**
   `/var/log` (unrotated app logs), `/var/log/journal` (`journalctl --vacuum-size=500M`), `/var/lib/docker` (`docker system df` then `docker system prune -a --volumes` — confirm with the owner first), `/tmp`, core dumps in `/var/lib/systemd/coredump`, old releases under `/opt/app/releases`, package caches (`apt clean`, `dnf clean all`).

5. **Buy time safely**
   Truncate rather than delete an active log: `sudo truncate -s 0 /var/log/app/app.log` keeps the inode so the writer does not lose its descriptor. `rm` on an open log frees nothing until restart.

6. **Verify and prevent**
   Re-check `df -h`, then fix the cause: logrotate config, a `SizeMaxUse=` in journald, a retention cron. Write it into the handover so it does not recur next month.

Never `rm -rf` to free space under pressure

At 2am on someone else's production host, `rm -rf /var/lib/<something>` is how an incident becomes a data-loss incident. Identify, confirm ownership with the customer, prefer truncate and rotate, and keep a record of everything you removed.

Logs: Where They Live and How to Read Them

Logs are the only evidence you have about what happened before you arrived. Knowing the layout of a stranger's logging setup in under a minute is a differentiating skill.

Path	Debian/Ubuntu	RHEL family	Contains
General syslog	`/var/log/syslog`	`/var/log/messages`	Everything not routed elsewhere
Auth	`/var/log/auth.log`	`/var/log/secure`	SSH logins, sudo, PAM, failed attempts
Kernel	`/var/log/kern.log`, `dmesg`, `journalctl -k`		OOM kills, disk errors, network drivers
Boot	`journalctl -b`, `/var/log/boot.log`		Why a service did not start at boot
Journal	`/var/log/journal/` (binary) or `/run/log/journal` (volatile)		All systemd-captured stdout/stderr
Packages	`/var/log/dpkg.log`, `apt/history.log`	`/var/log/dnf.log`, `rpm -qa --last`	"What changed last Tuesday?"
Web	`/var/log/nginx/`, `/var/log/httpd/`		access + error logs
Containers	`/var/lib/docker/containers/<id>/<id>-json.log`		What `docker logs` reads

BASH

```bash
# The "what changed" question, answered three ways
grep -E ' (install|upgrade|remove) ' /var/log/dpkg.log | tail -30
rpm -qa --last | head -30
journalctl --since '3 days ago' -u '*' -p warning --no-pager | head -50
ls -lt /etc | head -20            # most recently modified config files
sudo find /etc -mtime -7 -type f  # anything edited in the last week
```

Tailing and filtering under pressure

BASH

```bash
tail -f /var/log/app/app.log
tail -F /var/log/app/app.log              # survives logrotate (reopens by name)
tail -n 500 app.log | grep -i error
multitail /var/log/a.log /var/log/b.log   # if available
journalctl -u api -u worker -f            # follow several units at once
journalctl -f | grep --line-buffered ERROR # --line-buffered or grep buffers 4 KB firs
stdbuf -oL -eL ./noisy_tool | grep WARN   # force line buffering on a stubborn progra
less +F /var/log/app/app.log              # Ctrl-C to stop following, / to search, F
```

less beats tail for investigation

In `less`: `/pattern` searches forward, `?pattern` backward, `n` / `N` cycles, `&pattern` shows *only* matching lines (like an interactive grep), `G` jumps to the end, `g` to the start, `-S` stops line wrapping. `&` alone is worth learning — it turns a 2 GB log into a filtered view instantly without re-reading the file.

logrotate

TEXT

```
# /etc/logrotate.d/inference-api
/var/log/inference-api/*.log {
    daily
    rotate 14
    size 100M
    compress
    delaycompress
    missingok
    notifempty
    create 0640 svc_api app
    sharedscripts
    postrotate
        systemctl kill -s HUP inference-api.service 2>/dev/null || true
    endscript
}
```

BASH

```
sudo logrotate -d /etc/logrotate.d/inference-api    # DEBUG: show what would happen
sudo logrotate -f /etc/logrotate.d/inference-api    # force a rotation now
cat /var/lib/logrotate/status                        # when each file last rotated
```

Use `copytruncate` only if the application cannot reopen its log on SIGHUP — it is simpler but races, and you can lose lines written between the copy and the truncate.

Working Remotely for Days: tmux, Dotfiles, Persistence

Engagements are measured in weeks. Your VPN will drop mid-migration. Your laptop will sleep. A terminal multiplexer turns those from incidents into non-events, and a portable dotfile setup means your tenth customer host feels like your first.

tmux

BASH

```
tmux new -s acme              # named session
tmux ls                        # list sessions
tmux attach -t acme            # reattach after the VPN drops
tmux attach -d -t acme         # detach other clients first
tmux kill-session -t acme
tmux new -s acme -d 'tail -F /var/log/app/app.log'   # start detached, running somethi
```

Keys (prefix +)	Action	Keys	Action
`d`	Detach (session keeps running)	`c`	New window
`"` / `%`	Split horizontal / vertical	`0` – `9`	Jump to window
Arrow keys	Move between panes	`z`	Zoom pane to full screen (toggle)
`[`	Copy/scroll mode (q exits)	`,`	Rename window
`x`	Kill pane	`s`	Session picker

TEXT

```
# ~/.tmux.conf - portable, minimal, no plugins required
set -g mouse on
set -g history-limit 100000
set -g base-index 1
setw -g pane-base-index 1
set -g status-right '#(whoami)@#H  %Y-%m-%d %H:%M'
bind r source-file ~/.tmux.conf \; display "reloaded"
bind | split-window -h
bind - split-window -v
set -sg escape-time 10
```

GNU screen is the fallback

Some locked-down images ship `screen` but not `tmux`. Minimum viable: `screen -S acme` to start, `Ctrl`+`A` `d` to detach, `screen -r acme` to reattach, `screen -ls` to list. Know both.

Persistence without a multiplexer

BASH

```bash
nohup ./long_migration.sh > migration.log 2>&1 &
echo $! > /tmp/migration.pid
disown                                        # detach from this shell

setsid ./job.sh > job.log 2>&1 < /dev/null &  # fully detached, new session leader

# Best option when systemd is available and you have the rights:
systemd-run --user --unit=migration --working-directory=/opt/app ./long_migration.sh
journalctl --user -u migration -f
```

`systemd-run` is the cleanest: the job survives logout, is supervised, gets a cgroup, and its output goes to the journal automatically — no stray log files to clean up.

Dotfiles that travel

BASH

```bash
# A single idempotent bootstrap you can paste into any customer shell
cat >> ~/.bashrc <<'EOF'
export HISTSIZE=50000 HISTFILESIZE=100000
export HISTCONTROL=ignoreboth:erasedups
export HISTTIMEFORMAT='%F %T  '
shopt -s histappend checkwinsize globstar cdspell
export EDITOR=vim PAGER=less LESS='-R -S -i -M'
alias ll='ls -alhF' la='ls -A' l='ls -CF'
alias ..='cd ..' ...='cd ../..'
alias grep='grep --color=auto'
alias df='df -h' du='du -h'
alias ports='ss -tulpn'
alias psg='ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -20'
PS1='\[\e[1;31m\]\u@\h\[\e[0m\]:\[\e[1;34m\]\w\[\e[0m\]\$ '    # red user@host = NOT my
EOF
```

✦ **Make customer hosts look different**

A red prompt on every host that is not yours is the cheapest possible guard against running a destructive command in the wrong window. Pair it with a distinctive tmux status bar per engagement. People who have pasted a `DROP TABLE` into the wrong pane never skip this again.

⚠ **Ask before you install**

Cloning a dotfiles repo onto a customer host means pulling code from the internet onto their system. On a regulated engagement that may be a policy violation. Prefer a short inline heredoc like the one above, confined to your own home directory, and mention it in your handover.

The FDE Field Kit

One table to keep open, and one procedure to run every single time you land on a host you have never seen.

Command cheat sheet

Question	Command
What OS and version is this?	`cat /etc/os-release; uname -a; hostnamectl`
Who am I and what can I do?	`id; sudo -l; groups`
What is running?	`systemctl list-units --type=service --state=running`
What is broken?	`systemctl --failed; journalctl -p err -b --no-pager | tail -50`
What is listening?	`ss -tulpn`
What is eating CPU/RAM?	`ps -eo pid,user,%cpu,%mem,rss,cmd --sort=-%cpu | head`
Where did the disk go?	`df -h; df -i; du -xh --max-depth=1 /var | sort -h | tail`
Can I reach X?	`timeout 3 bash -c '>/dev/tcp/host/port' && echo open`
Why is DNS wrong?	`cat /etc/resolv.conf; dig +short host; getent hosts host`
Is it the proxy?	`env | grep -i proxy; curl -v --noproxy '*' URL`
Is it the certificate?	`openssl s_client -connect host:443 -servername host </dev/null`
What changed recently?	`rpm -qa --last | head` / `tail /var/log/dpkg.log`; `find /etc -mtime -7`
Who logged in?	`last -20; lastb -20; journalctl -u sshd --since -1d`
What is this process doing?	`strace -f -p PID`; `cat /proc/PID/{status,limits,environ}`
Which files are open?	`lsof -p PID; lsof -i :PORT`
Find config/secret files	`find / -xdev -name '*.conf' -newer /etc/hostname 2>/dev/null`
Top error strings	`grep -oE 'ERROR.*' app.log | sort | uniq -c | sort -rn | head`

Status code histogram	`awk '{c[$9]++} END{for(k in c) print c[k],k}' access.log | sort -rn`
Keep a session alive	`tmux new -s work` / `tmux attach -t work`
Diff two hosts	`diff <(ssh a 'cmd') <(ssh b 'cmd')`

First 15 minutes on an unfamiliar server

1. **Start a recorded, persistent session**
   `tmux new -s acme`, then `script -a ~/session-$(date +%F).log` if policy allows. You will want the transcript later, and so will the customer's ticket.

2. **Identify the box**
   `hostnamectl; cat /etc/os-release; uname -r; nproc; free -h; df -h`. Write distro, version, kernel, core count, RAM and free disk into your notes before touching anything.

3. **Identify yourself**
   `id; sudo -l; groups`. Know your blast radius before you need it, not while an outage is running. If `sudo -l` prompts for a password you do not have, find that out now.

4. **Check health**
   `uptime; systemctl --failed; journalctl -p err -b --no-pager | tail -40; dmesg -T | tail -30`. Note anything already broken so you are not blamed for it.

5. **Map the network posture**
   `ip -brief addr; ip route; cat /etc/resolv.conf; env | grep -i proxy; ss -tulpn`. Determine immediately whether there is egress: `timeout 5 curl -sI https://example.com | head -1`.

6. **Find the application**
   `systemctl list-units --type=service --state=running`, then `systemctl cat <unit>` for the exact `ExecStart`, user, working directory and environment file. That one command tells you where the code, config and logs are.

7. **Locate the evidence**
   `ls -lt /var/log | head`; identify the app's log directory and whether the journal is persistent (`ls /var/log/journal`).

8. **Check what is installed**
   `for t in git curl jq python3 docker kubectl podman rsync tmux; do command -v $t || echo "MISSING $t"; done`. Decide now whether you need to request packages or bring

static binaries.

9. **Establish a workspace**
`mkdir -p ~/work && cd ~/work`. Keep everything you create in one directory so cleanup at the end of the engagement is a single `rm -rf` you can describe precisely.

10. **Snapshot before you change anything**
`sudo cp -a /etc/app /etc/app.bak-$(date +%F)` for any config you plan to edit; `rpm -qa | sort > ~/work/packages-before.txt`. Being able to diff and revert is what separates a consultant from a liability.

✓ **Leave a trail**
Keep a running `~/work/CHANGELOG.md`: timestamp, command, reason, rollback. At the end of the engagement that file becomes the handover document, and during the engagement it is what you read out when someone asks "what did you change at 14:30?"

Interview Questions

Expect these in any FDE, SRE, platform or solutions-engineering loop. The answers below are the depth an interviewer is listening for — mechanism first, then the practical consequence.

> **Walk me through what happens when you type `ls -l | grep foo` and press Enter.**
> Bash parses the line into a pipeline of two commands and performs expansions (brace, tilde, parameter, command substitution, arithmetic, then word splitting and globbing) on each. It creates a pipe with `pipe()`, giving a read and a write file descriptor. It forks twice. In the first child it dups the pipe's write end onto fd 1, closes the unused read end, and calls `execve("/bin/ls", …)` after resolving `ls` through PATH. In the second child it dups the read end onto fd 0 and execs `grep`. The parent closes both pipe ends — important, or `grep` would never see EOF — and waits. Both processes run concurrently; `ls` blocks when the 64 KB pipe buffer fills until `grep` drains it. The shell reports the exit status of the last command unless `pipefail` is set. Crucially, globbing happened in the *shell* before exec, which is why you quote patterns you want the command to interpret.

> **A customer says "the API can't reach the database." You have SSH on the API host. What do you do?**
> I work up the stack and stop at the first layer that fails. First, does the name resolve — `getent hosts db.internal` and `dig +short db.internal`, checking both because the application goes through NSS while dig does not, and a stale `/etc/hosts` entry would show the difference. Second, is there a route — `ip route get <ip>`. Third, does TCP connect — `timeout 3 bash -c`

`'>/dev/tcp/db.internal/5432'`, which needs no tools. If that hangs, a firewall is dropping; if it is refused immediately, nothing is listening or a firewall is rejecting. Fourth, if TLS is involved, `openssl s_client -connect db:5432 -starttls postgres` to check certificate and expiry. Fifth, application layer: credentials, the actual connection string the app is using (`tr '\0' '\n' < /proc/<pid>/environ`), and whether the connection pool is exhausted. In parallel I check `env | grep -i proxy`, because an internal host missing from `NO_PROXY` produces exactly this symptom with a 30-second hang. I also check from the database side with `ss -tn state established 'sport = :5432'` to see whether connections are arriving at all — that single fact tells the customer's network team whether the problem is in front of or behind the firewall.

`df` says the disk is 100% full but `du` only accounts for 40%. Explain.

Almost certainly a deleted-but-still-open file. When a process holds a file descriptor and the file is unlinked, the directory entry disappears — so `du`, which walks the directory tree, cannot see it — but the inode and its blocks stay allocated until the last descriptor closes. A log file rotated with `rm` instead of truncate, while the writer still holds it open, is the classic cause. I confirm with `sudo lsof +L1` or `lsof | grep deleted`, which shows the holding PID and the size. The fix is to restart or SIGHUP the process so it reopens its log, or truncate through the descriptor with `: > /proc/<pid>/fd/<n>` if a restart is not acceptable. Two other possibilities worth ruling out: inode exhaustion (`df -i`), which produces "No space left on device" with plenty of free bytes; and a filesystem mounted *over* a populated directory, hiding the data underneath from `du` but not from the underlying filesystem's accounting.

What does `set -euo pipefail` do, and where does it fail you?

`-e` exits on the first command returning non-zero, `-u` treats expansion of an unset variable as an error, and `-o pipefail` makes a pipeline return the status of the rightmost failing stage rather than the last stage. Together they turn silent failures into loud ones. The gaps: `-e` is deliberately suppressed inside conditions, so `if cmd; then`, `cmd && …`, `cmd || …` and `! cmd` do not abort — and that suppression propagates into functions called from those contexts, which surprises people. It also does not fire on a command whose failure is in the middle of a `&&` chain that ultimately succeeds. `-u` breaks on `"${arr[@]}"` for an empty array in bash below 4.4, so you write `"${arr[@]:-}"`. `pipefail` makes `producer | head -1` fail with SIGPIPE/141 once head exits, which you have to explicitly tolerate. So I use all three as the default, and I pair them with an `ERR` trap that prints `$LINENO` and `$BASH_COMMAND`, plus an `EXIT` trap for cleanup — because `-e` tells you that something failed but not what, and on a customer system that distinction matters.

Difference between SIGTERM and SIGKILL, and why does it matter in production?

SIGTERM (15) is a request: the process can install a handler, finish in-flight requests, flush buffers, close database connections, deregister from a load balancer, and exit cleanly. SIGKILL (9) is executed by the kernel — the process never runs another instruction, no handler fires, no buffer is flushed, no lock is released. The practical consequences: a web server killed with -9 drops in-flight connections and produces a burst of 502s; Postgres killed with -9 enters crash recovery on next start; a process holding a lock file leaves it stale and blocks the next start. The correct pattern is escalation — send

TERM, wait a bounded grace period, then KILL only if it is still alive. That is precisely what systemd does (`KillSignal=` , `TimeoutStopSec=` , then SIGKILL) and what Kubernetes does (`preStop` hook, SIGTERM, `terminationGracePeriodSeconds` , SIGKILL). Note also that SIGKILL cannot interrupt a process in uninterruptible sleep (state D) — if a process is blocked on a hung NFS mount, even -9 will not remove it, which is a useful diagnostic in itself.

You need to reach a Postgres that only a bastion can see, from a GUI tool on your laptop. How?

An SSH local port forward: `ssh -N -L 5432:db.internal:5432 bastion` , then point the GUI at `127.0.0.1:5432` . The `-N` means no remote command, just the tunnel. The hostname `db.internal` is resolved *on the bastion*, which is the point — my laptop does not need to resolve or route to it. If the bastion itself is behind another hop I add `-J jump1` , and I put the whole thing in `~/.ssh/config` with `ProxyJump` and `LocalForward` so it is one word to reconnect. If I need many internal services, `ssh -D 1080` gives a SOCKS5 proxy and I configure the browser or use `curl --socks5-hostname` . Two caveats I would raise with the customer: forwarding must be permitted (`AllowTcpForwarding yes` in sshd_config, and many hardened bastions disable it), and this is a data path into a production database, so it needs to be sanctioned, time-boxed, and ideally read-only credentials. I would not leave a standing tunnel running.

How do you find what is consuming memory on a host, and how do you prove an OOM kill happened?

I start with `free -h` and read the **available** column, not free — Linux uses idle RAM for page cache and reclaims it on demand, so low "free" is normal. Then `ps -eo pid,user,rss,vsz,comm --sort=-rss | head` for the biggest resident sets; RSS over-counts shared pages across processes, so for a fork-heavy app like gunicorn or Chrome I prefer `smem` and its PSS figure. For a proof of an OOM kill I check the kernel ring buffer: `dmesg -T | grep -i 'killed process'` or `journalctl -k --since -24h | grep -i oom` . The kernel prints the victim's PID, name, `total-vm` and `anon-rss` at kill time, which both confirms the event and gives you the size it reached. In containers and under systemd it may be a cgroup limit rather than global pressure — then I check `systemctl show app -p MemoryMax -p MemoryCurrent` or `/sys/fs/cgroup/ … /memory.events` for a non-zero `oom_kill` counter. That distinction matters: global OOM means the host is undersized, cgroup OOM means the limit is wrong or the app leaks.

Hard link versus symbolic link.

A hard link is an additional directory entry pointing at the same inode. There is no "original" — the inode has a link count, the data persists until the count reaches zero and no process holds it open. Hard links cannot cross filesystems (inode numbers are only meaningful within one) and by convention cannot be made to directories, because that would permit cycles the tree-walking tools cannot handle. A symbolic link is a distinct inode whose content is a path string, resolved at access time. It can cross filesystems, can point at directories, can point at something that does not exist yet (dangling), and tools can choose to follow it or not — hence `ls -L` , `cp -P` , `find -L` . Operationally I use symlinks for release management: deploy to `releases/<version>` and atomically repoint `current` with `ln -sfn` , so rollback is one syscall. I use hard links for

deduplicated backups and for rsync's `--link-dest`. And I remember that `rm` of a symlink removes the link, while `rm symlink/` with a trailing slash may follow it — a distinction that has deleted real data.

What is the difference between `>file 2>&1` and `2>&1 >file`?

Redirections are processed left to right and `2>&1` means "point fd 2 at whatever fd 1 currently points at" — it is a copy of the current target, not a permanent alias. In `>file 2>&1`, stdout is first pointed at the file, then stderr is pointed at the same file; both end up in the file. In `2>&1 >file`, stderr is first pointed at the terminal (where stdout currently points), and only then is stdout moved to the file — so errors still appear on your screen and only normal output is captured. The second form is occasionally what you want: `cmd 2>&1 >/dev/null | grep -i error` pipes stderr while discarding stdout, because the pipe is established before the redirections are applied. This trips people up constantly when they wonder why their captured log file contains no error messages.

A systemd service fails to start. Walk me through your diagnosis.

`systemctl status app` first — it gives the state, the exit code or signal, and the last few journal lines, which is often enough. Then `journalctl -u app -n 200 --no-pager` for the full output, and `journalctl -u app -b` if the failure was at boot. Next `systemctl cat app` to see the effective unit including drop-ins, and I check the obvious: does the `ExecStart` binary actually exist and is it executable (`Type=exec` catches this where `Type=simple` would not), does the `User=` exist, does `WorkingDirectory=` exist and is it readable by that user, does the `EnvironmentFile=` exist (without a leading `-` an absent file is fatal). Common non-obvious causes: the unit was edited but `systemctl daemon-reload` was never run; `Restart=always` hit `StartLimitBurst` and the unit is now refusing to start until `systemctl reset-failed`; a hardening directive like `ProtectSystem=strict` is blocking a write path not listed in `ReadWritePaths=`; or an SELinux denial, which I confirm with `ausearch -m avc -ts recent`. To isolate application from environment I run the exact `ExecStart` line manually as the service user with `sudo -u svc_app env -i …` — if it works interactively but not under systemd, the difference is environment, namespacing or permissions, and I have narrowed it to three candidates.

How would you install software on a host with no internet access?

First I establish what "no internet" means — often there is an internal mirror (Nexus, Artifactory, Satellite) that is the sanctioned path, and the right answer is to point the package manager at it rather than to smuggle files in. If it is genuinely air-gapped, I use a connected machine with the *same distro, version and architecture* — easiest as a container of the matching base image — and resolve the full dependency set there: `dnf download --resolve --alldeps`, or `apt-get install --download-only`, or `pip download --platform manylinux2014_x86_64 --only-binary=:all:`. For RPM I build a local repo with `createrepo_c` so dnf can resolve offline. I checksum the bundle, record the hash in the change ticket, transfer via the approved channel, and install with the remote repos explicitly disabled (`dnf --disablerepo='*' --enablerepo=offline`). Then I verify (`rpm -V`, a version flag, a smoke test) and document exactly what was added. Where possible I avoid the problem entirely by shipping a statically linked binary or a container image moved with `docker`

> `save` / `load` — one artifact, no dependency resolution, one approval. The non-technical half matters just as much: getting the artifact through the customer's security review usually takes longer than the installation does, so I start that conversation on day one.

> **What is the difference between a login shell and a non-login shell, and why does it break cron jobs?**
> A login shell reads `/etc/profile` and then the first existing of `~/.bash_profile`, `~/.bash_login`, `~/.profile`. An interactive non-login shell reads `/etc/bash.bashrc` and `~/.bashrc`. A non-interactive shell — which is what `ssh host 'cmd'`, cron, systemd and CI all use — reads neither; it only sources `$BASH_ENV` if that is set. So every PATH entry, alias, `nvm` / `pyenv` initialisation and proxy export you put in `.bashrc` is invisible to cron. The symptom is always the same: "command not found" or the wrong interpreter version, for a command that works perfectly when you SSH in and type it. The fix is to stop relying on shell initialisation in automated contexts — use absolute paths in crontabs and units, set `PATH=` explicitly at the top of the crontab, and put environment into a systemd `EnvironmentFile=` rather than a dotfile. Structurally, I keep environment in `~/.profile`, interactive-only settings in `~/.bashrc`, and have `~/.bash_profile` source `~/.bashrc` so interactive logins get both.

Practice Drills and a 14-Day Fluency Plan

Reading about the shell builds recognition; typing builds speed. Work through this on a throwaway VM or container (`docker run -it --rm ubuntu:22.04 bash` is enough for most of it) and time yourself. The goal is that none of these require thought under pressure.

Daily drills (15 minutes)

- [ ] Navigate to three directories using only `cd -`, `pushd` / `popd` and tab completion — no full paths typed
- [ ] Reconstruct yesterday's longest command from history using `Ctrl`+`R` only
- [ ] Write one pipeline that answers a question about a log file, in under 60 seconds
- [ ] Explain out loud what each flag in a command you copied actually does; look up any you cannot
- [ ] Pick one command you use daily and read its `--help` in full, looking for a flag you did not know

The 14-day plan

Day	Focus	Deliverable you must produce
1	Navigation, FHS, readline	Map `/etc`, `/var`, `/opt` on two different distros; list five differences
2	find + xargs	Find all files >10 MB modified in 7 days, excluding one subtree, and total their size
3	Permissions, umask, setgid	Build a shared group directory where new files are automatically group-writable
4	Processes, signals, lsof	Start a process, find its port with `lsof`, HUP it, then kill it by pattern
5	systemd units	Write, enable and debug a service unit for a Python HTTP server, with `MemoryMax` and auto-restart
6	systemd timers + journalctl	Convert a crontab entry to a timer; prove it caught up after a simulated downtime
7	Package management	Install nginx offline into a network-disabled container from artifacts you staged
8	SSH config + tunnels	Reach a container-only Postgres from your host through a two-hop `ProxyJump` chain
9	grep / sed / awk	Produce a top-10 endpoint latency report from a raw access log in one pipeline
10	jq	Turn a 5 MB Kubernetes JSON dump into a TSV of name, phase, restart count, node
11	Redirection, FDs, here-docs	Write a script that logs stdout and stderr to separate files while still printing both
12	Bash scripting	Write a 100-line script with arg parsing, traps, dry-run and rollback; pass `shellcheck` clean
13	Networking	Break DNS, break routing and break TLS in a lab, then diagnose each from scratch in under 3 minutes
14	Full simulation	Have someone sabotage a VM (full disk, failed unit, wrong permission, bad proxy); fix it with notes

Lab scenarios worth building

Scenario 1 — The service that will not start

Scenario 2 — Disk full at 2am

Scenario 3 — It cannot reach the API

Scenario 4 — Air-gapped deployment end to end

✓ **The standard to aim for**

You should be able to land on an unfamiliar Linux host and, without reaching for a search engine, determine the distro, your privileges, what is running, what is broken, what is listening, where the logs are, and whether you have egress — in under five minutes. Everything else in this guide is in service of that one capability.